NGÔN NGỮ C#

ThS. Trần Anh Dũng

fppt.com

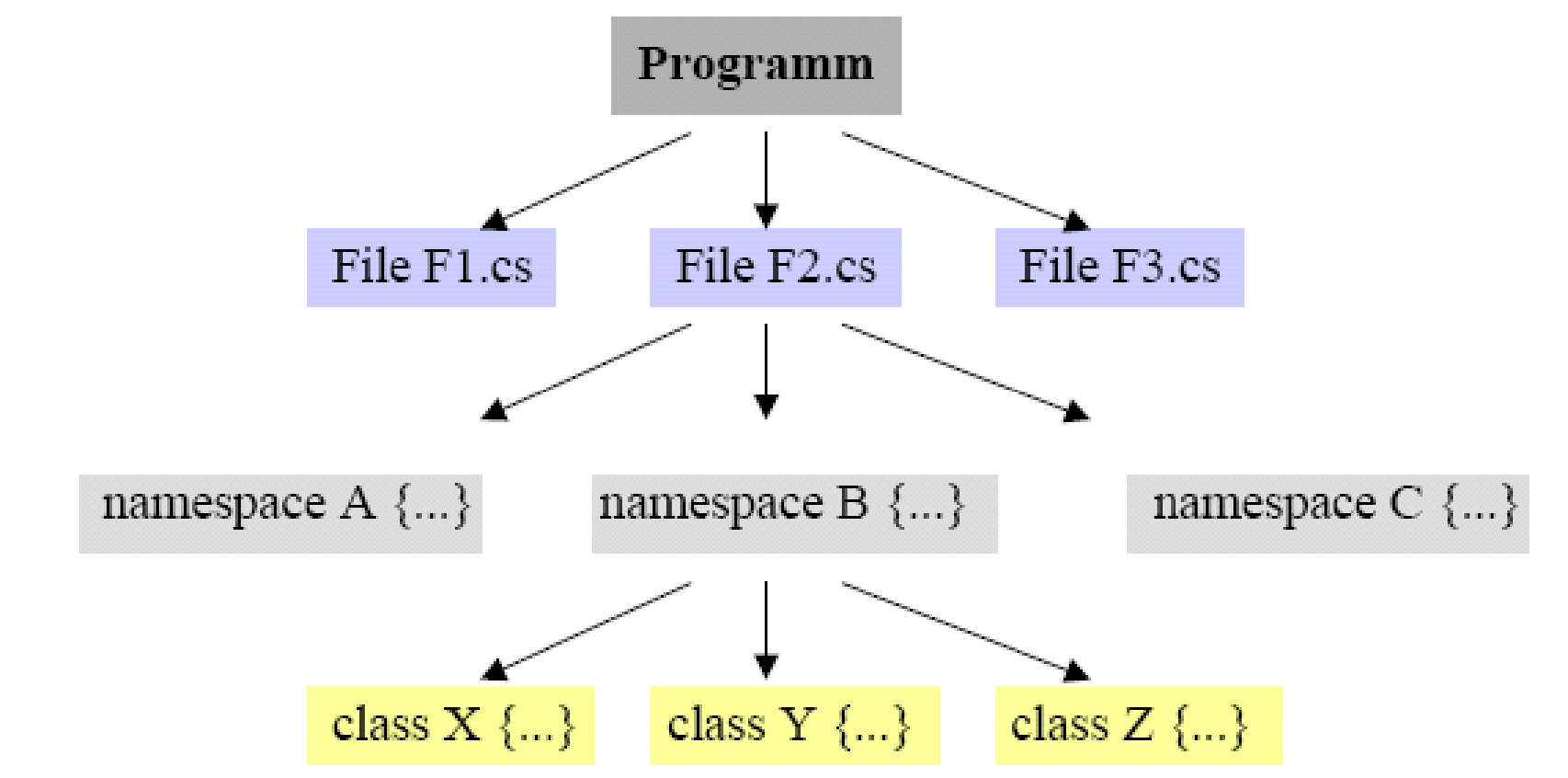
C#

- Ngôn ngữ lập trình “thuần” hướng đối tượng
- 70% Java, 10% C++, 5% Visual Basic, 15% mới
- Trình biên dịch C# là một trong những trình biên dịch hiệu quả nhất trong dòng sản phẩm .NET.

Đặc điểm của ngôn ngữ C#

- Có từ khóa khai báo dành cho thuộc tính (property)
- Cho phép tạo sưu liệu trực tiếp bên trong mã nguồn (dùng tool mã nguồn mở NDoc phát sinh ra sưu liệu)
- Hỗ trợ khái niệm interface (tương tự java)
- Cơ chế tự động dọn rác (tương tự java)
- Truyền tham số kiểu: in(ø), out, ref, param

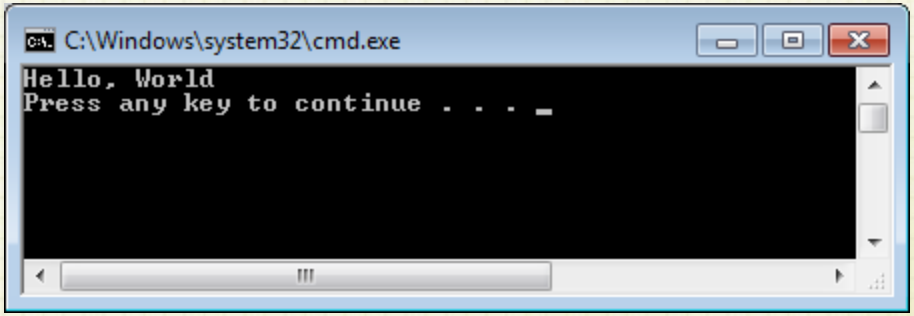
Cấu trúc chương trình C#



Hello World

```
using System;

class Hello
{
    public static void Main()
    {
        Console.WriteLine("Hello, World");
    }
}
```



Namespace

- Namespace cung cấp cho cách tổ chức quan hệ giữa các lớp và các kiểu khác.

```
namespace CustomerPhoneBookApp {
    using System;
    public struct Subscriber {
        // Code for struct here...
    }
}
```

Namespace

- Từ khóa using giúp giảm việc phải gõ những namespace trước các hàm hành vi hoặc thuộc tính

  using Wrox.ProCSharp;

- Ta có thể gán bí danh cho namespace

  Cú pháp:

  using *alias = NamespaceName;*

```
namespace com.carnage4life {
   public class MyClass {
      int x;
      void doStuff(){}
   }
}
```

C# Code

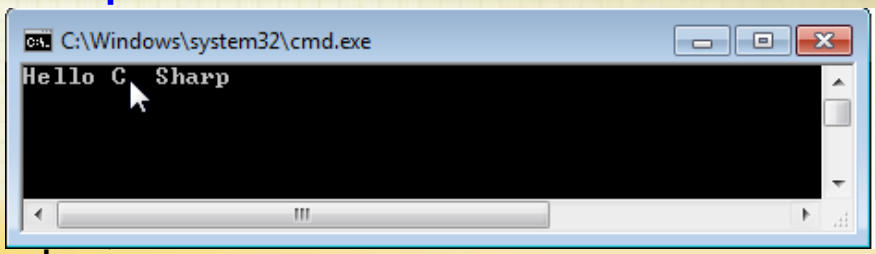
```
package com.carnage4life;
public class MyClass {
  int x;
  void doStuff(){}
}
```

Java Code

Ví dụ

```
/* Chương trình cơ bản của C#*/
class Hello{
        static void Main(string[] args)
        {
            System.Console.WriteLine("Hello C  Sharp");
            System.Console.ReadLine();
        }
}
```

- Để biên dịch từng Class, có thể sử dụng tập tin csc.exe trong cửa sổ Command Prompt với khai báo như sau: D:\csc CSharp\ Hello.cs



Ví dụ

```
01      /* Chương trình cơ bản của C#*/
02      using System;
03      class Hello{
04              static void Main(string[] args){
05                  Console.WriteLine("Hello C  Sharp");
06                  Console.ReadLine();
07              }
08      }
```

Ví dụ

```
01      /* Chương trình cơ bản của C#*/
02      using Con=System.Console;
03      class Hello{
04              static void Main(string[] args){
05                  Con.WriteLine("Hello C  Sharp");
06                  Con.ReadLine();
07              }
08      }
```

Console I/O

- Để đọc ký tự văn bản từ cửa sổ console
  - Console.Read() giá trị trả về là int

```
int n = Console.Read();
Console.WriteLine((char)n);
```

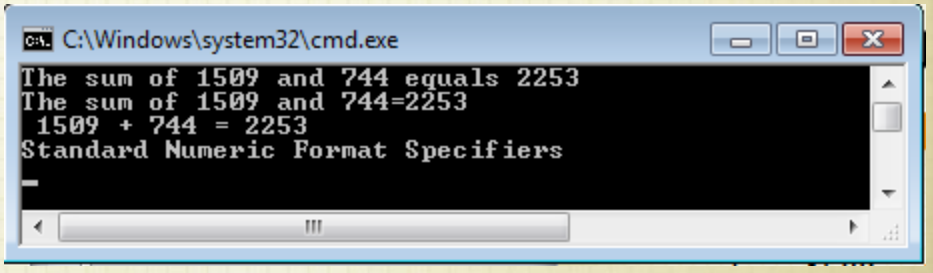
  - Console.ReadLine() giá trị trả về là string

```
string s = Console.ReadLine();
Console.WriteLine(s);
```

- Để xuất chuỗi ký tự dùng
  - Console.Write() / Console.WriteLine()

Console.WriteLine

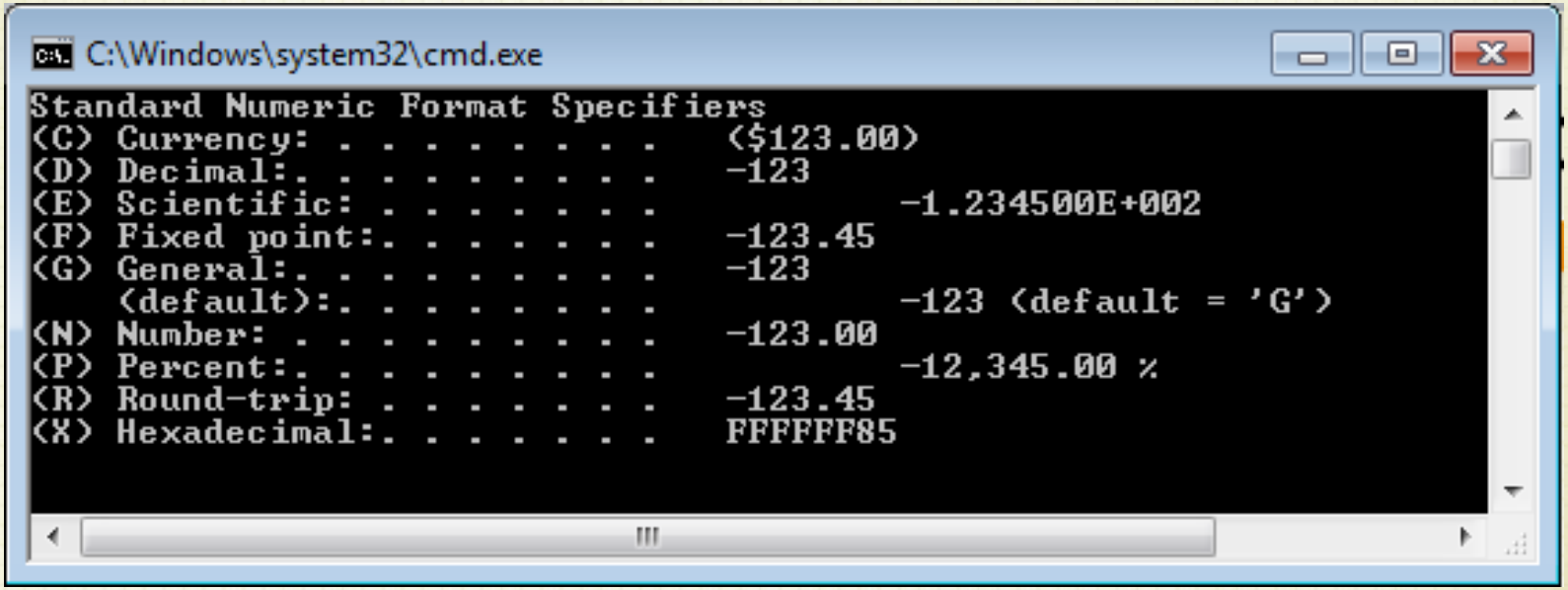
```
public static void Main() {
        int a = 1509; int b = 744; int c = a + b;
        Console.Write("The sum of ");
        Console.Write(a);
        Console.Write(" and ") ;
        Console.Write(b);
        Console.Write(" equals ");
        Console.WriteLine(c);
        Console.WriteLine("The sum of " + a + " and " +
  b + "="+c) ;
        Console.WriteLine(" {0} + {1} = {2}", a, b, c);
        Console.ReadLine();
}
```



Console.WriteLine

```
Console.WriteLine("Standard Numeric Format
Specifiers");
Console.WriteLine(
        "(C) Currency: . . . . . . . .      {0:C}\n" +
        "(D) Decimal:. . . . . . . . .      {0:D}\n" +
        "(E) Scientific: . . . . . . .      {1:E}\n" +
        "(F) Fixed point:. . . . . . .      {1:F}\n" +
        "(G) General:. . . . . . . . .      {0:G}\n" +
        "    (default):. . . . . . . .      {0}
(default = 'G')\n" +
        "(N) Number: . . . . . . . . .      {0:N}\n" +
        "(P) Percent:. . . . . . . . .      {1:P}\n" +
        "(R) Round-trip: . . . . . . .      {1:R}\n" +
        "(X) Hexadecimal:. . . . . . .      {0:X}\n",
        -123, -123.45f);
```

Console.WriteLine



Console.WriteLine

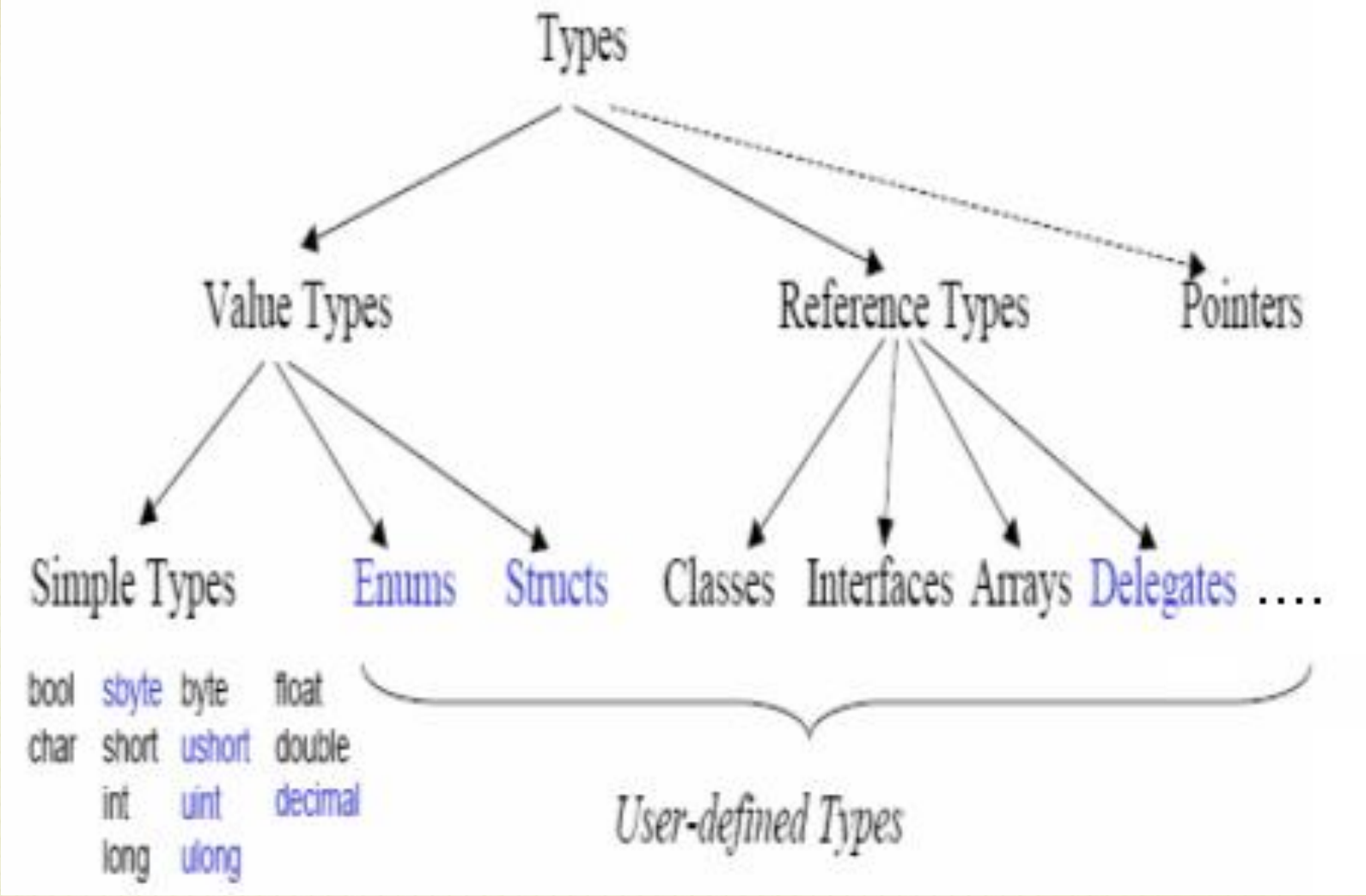
```
Console.WriteLine("Standard DateTime Format Specifiers");
Console.WriteLine(
        "(d) Short date: . . . . . . .                {0:d}\n" +
        "(D) Long date:. . . . . . . .        {0:D}\n" +
        "(t) Short time: . . . . . . .                {0:t}\n" +
        "(T) Long time:. . . . . . . .                {0:T}\n" +
        "(f) Full date/short time: . .        {0:f}\n" +
        "(F) Full date/long time:. . .        {0:F}\n" +
        "(g) General date/short time:.        {0:g}\n" +
        "(G) General date/long time: .        {0:G}\n" +
        "    (default):. . . . . . . .                {0} (default
= 'G')\n" +
        "(M) Month:. . . . . . . . . .                {0:M}\n" +
        "(R) RFC1123:. . . . . . . . .        {0:R}\n" +
        "(s) Sortable: . . . . . . . .                {0:s}\n" +
        "(u) Universal sortable: . . .        {0:u} (invariant)\n"
+
        "(U) Universal sortable: . . .        {0:U}\n" +
        "(Y) Year: . . . . . . . . . .                {0:Y}\n",
        thisDate);
```

Console.WriteLine

C:\Windows\system32\cmd.exe

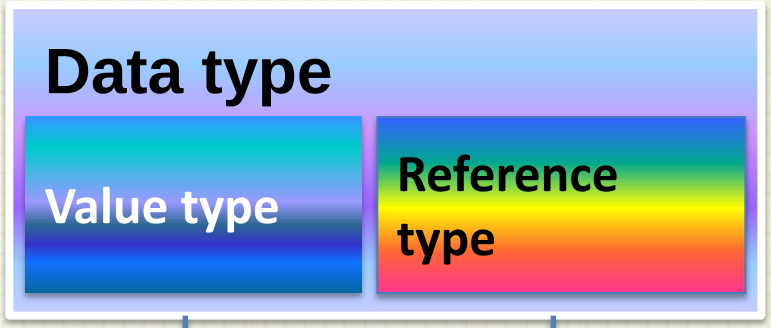
```
Standard DateTime Format Specifiers
(d) Short date: . . . . . .          11/20/2011
(D) Long date:. . . . . . . .   Sunday, November 20, 2011
(t) Short time: . . . . . . .        5:54 PM
(T) Long time:. . . . . . . .        5:54:04 PM
(f) Full date/short time: . .   Sunday, November 20, 2011 5:54 PM
(F) Full date/long time:. . .   Sunday, November 20, 2011 5:54:04 PM
(g) General date/short time:.   11/20/2011 5:54 PM
(G) General date/long time: .   11/20/2011 5:54:04 PM
    (default):. . . . . . . .        11/20/2011 5:54:04 PM (default = 'G')
(M) Month:. . . . . . . . . .        November 20
(R) RFC1123:. . . . . . . . .   Sun, 20 Nov 2011 17:54:04 GMT
(s) Sortable: . . . . . . . .        2011-11-20T17:54:04
(u) Universal sortable: . . .   2011-11-20 17:54:04Z (invariant)
(U) Universal sortable: . . .   Sunday, November 20, 2011 10:54:04 AM
(Y) Year: . . . . . . . . . .        November, 2011
```

Kiểu dữ liệu trong C#



Phân loại kiểu dữ liệu

- Phân loại theo cách thức lưu trữ dữ liệu




```
int num;
long count;
```

```
Object obj = new Object();
String str = “reference type”;
```

Value Type

- Chứa giá trị trực tiếp
- Không thể null
  - Phải chứa giá trị xác định
- Bao gồm
  - Primitive type
    - double, char, int, float,...
  - Enum
  - struct

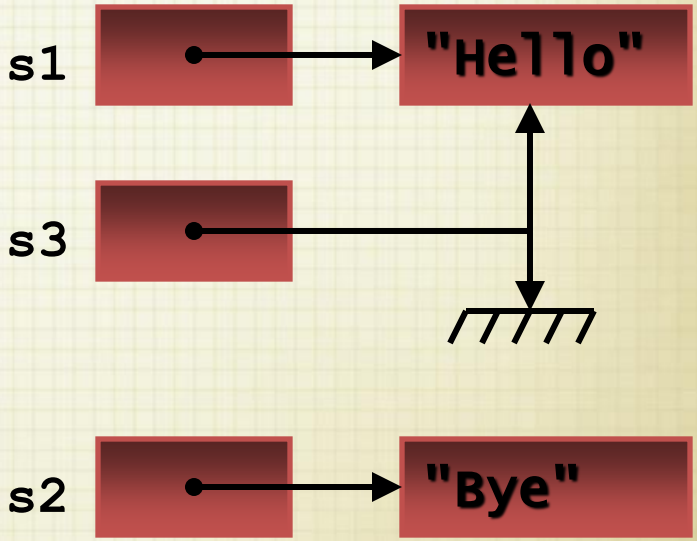
```
int i = 59;
double x = 7.83;
int a = i;
```

i	59
x	7.83
a	59

Reference Type

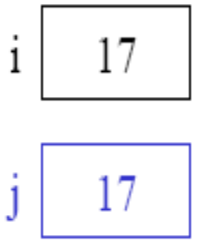
- Tham chiếu đến nơi chứa dữ liệu
- Có thể null
  - null: không chỉ tới bất kỳ đâu
- Bao gồm
  - Class
    - string, object
  - Interface
  - Array
  - Delegate

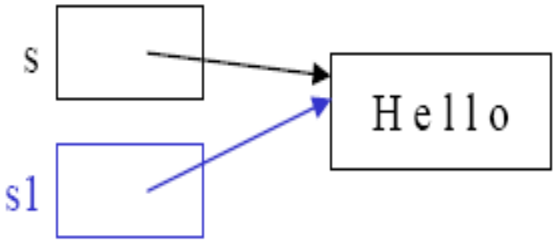
```
string s1 = "Hello";
string s2 = "Bye";
string s3;
s3 = s1;
```



Value Type vs. Reference Type

	Kiểu giá trị	Kiểu tham chiếu
Biến chứa	giá trị	tham chiếu
Được lưu trữ ở	stack	heap
Giá trị khởi tạo	0, false, '\0'	null
Lệnh gán	sao chép giá trị	sao chép tham chiếu
Ví dụ	int i = 17; int j = i;	string s = "Hello"; string s1 = s;





Kiểu dữ liệu định sẵn

Kiểu C#	Số byte	Kiểu .NET	Mô tả
byte	1	Byte	Số nguyên dương không dấu từ 0-255
char	2	Char	Kí tự Unicode
bool	1	Boolean	Giá trị logic true/ false
sbyte	1	Sbyte	Số nguyên có dấu (từ -128 đến 127)
short	2	Int16	Số nguyên có dấu giá trị từ -32768 đến 32767
ushort	2	UInt16	Số nguyên không dấu 0 – 65.535

Kiểu dữ liệu định sẵn

Kiểu C#	Số byte	Kiểu .NET	Mô tả
int	4	Int32	Số nguyên có dấu - 2.147.483.647 đến 2.147.483.647
uint	4	Uint32	Số nguyên không dấu 0 – 4.294.967.295
float	4	Single	Kiểu dấu chấm động, 3,4E-38 đến 3,4E+38, với 7 chữ số có nghĩa..
double	8	Double	Kiểu dấu chấm động có độ chính xác gấp đôi 1,7E-308 đến 1,7E+308 với 15,16 chữ số có nghĩa.

Kiểu dữ liệu định sẵn

Kiểu C#	Số byte	Kiểu .NET	Mô tả
decimal	8	Decimal	Có độ chính xác đến 28 con số dùng trong tính toán tài chính phải có hậu tố “m” hay “M” theo sau giá trị
long	8	Int64	Kiểu số nguyên có dấu -9.223.370.036.854.775.808 đến 9.223.372.036.854.775.807
ulong	8	Uint64	Số nguyên không dấu từ 0 đến 0xffffffffffffffff

Kiểu dữ liệu định sẵn

- Console.WriteLine("sbyte:{0} to {1} ",sbyte.MinValue,sbyte.MaxValue);
- Console.WriteLine("byte:{0} to {1}", byte.MinValue, byte.MaxValue);
- Console.WriteLine("short:{0} to {1}", short.MinValue, short.MaxValue);
- Console.WriteLine("ushort:{0} to {1}", ushort.MinValue,
  ushort.MaxValue);
- Console.WriteLine("int:{0} to {1}", int.MinValue, int.MaxValue);
- Console.WriteLine("long:{0} to {1}", long.MinValue, long.MaxValue);
- Console.WriteLine("decimal:{0} to {1}", decimal.MinValue,
  decimal.MaxValue);
- Console.ReadLine();

Kiểu dữ liệu định sẵn

C:\Windows\system32\cmd.exe

```
sbyte:-128 to 127
byte:0 to 255
short:-32768 to 32767
ushort:0 to 65535
int:-2147483648 to 2147483647
long:-9223372036854775808 to 9223372036854775807
decimal:-79228162514264337593543950335 to 79228162514264337593543950335
```

Constant, readonly

- Constant
  - const int x =5;
- Readonly
  - readonly int x;
- const:
  - Một hằng là một biến nhưng trị không thay đổi. Phải được gán giá trị khi khai báo.
- readonly:
  - Không cần khởi tạo trước, khi gán giá trị thì sau đó không thay đổi được.

Chuyển đổi kiểu dữ liệu

- Chuyển đổi dữ liệu là cho phép một biểu thức của kiểu dữ liệu này được xem xét như một kiểu dữ liệu khác.
- Chuyển đổi có thể: ẩn – ngầm định (implicit) hay tường minh (explicit),
- Ví dụ:

```
int a = 123;
long b = a;          // từ int sang long
int c = (int) b;     // từ long sang int
```

Using Convert class

- Thường dùng khi cần chuyển đổi giữa các kiểu không có liên hệ với nhau.
  - Convert.ToDataType(SourceValue)
- Ví dụ: Chuyển từ chuỗi sang số thực

```
string s1 = "56.8";
string s2 = "95";
double x = Convert.ToDouble(s1);     // x = 56.8
int i = Convert.ToInt32(s2);         // i = 95
byte j = Convert.ToByte(x);          // j = 56
```

Lớp Convert

Phương thức	Ý nghĩa
ToBoolean	Chuyển một giá trị sang giá trị Boolean
ToByte	Chuyển một giá trị sang giá trị số nguyên 8-bit không dấu
ToChar	Chuyển một giá trị sang giá trị ký tự unicode
ToDateTime	Chuyển một giá trị sang giá trị DateTime.
ToDecimal	Chuyển một giá trị sang giá trị Decimal.
ToDouble	Chuyển một giá trị sang giá trị số thực có độ chính xác gấp đôi 8 byte
ToInt16	Chuyển một giá trị sang giá trị số nguyên 16-bit có dấu
ToInt32	Chuyển một giá trị sang giá trị só nguyên 32-bit có dấu
ToInt64	Chuyển một giá trị sang giá trị số nguyên 64-bit có dấu
ToSByte	Chuyển một giá trị sang giá trị số nguyên 8-bit có dấu
ToSingle	Chuyển một giá trị sang giá trị số thực có độ chính xác đơn
ToString	Chuyển một giá trị sang giá trị một chuỗi
ToUInt16	Chuyển một giá trị sang giá trị số nguyên 16-bit không dấu
ToUInt32	Chuyển một giá trị sang giá trị số nguyên 32-bit không dấu
ToUInt64	Chuyển một giá trị sang giá trị số nguyên 64-bit không dấu

Enum(eration) – kiểu tập hợp

```
enum Days {Sat, Sun, Mon, Tue, Wed, Thu, Fri};
…
Days d = Days.Mon;
…
switch (d) {
   case Days.Tue: …
   case Days.Wed: …
}
```

Rõ hơn cách dùng hằng truyền thống của C

```
const int Sat = 1;
…
const int Fri = 6;
```

struct

- struct: value type (class: reference type)
- Dùng cho các đối tượng “nhỏ” như Point, Rectangle, Color,…

```
public struct MyPoint {
    public int x, y;
    public MyPoint(int p1, int p2) {
        x = p1;
        y = p2;
    }
}
```

Box và Unbox

- Đổi qua lại giữa value type và reference type.
- Box: value => reference (object).
- Thường dùng trong các hàm, cấu trúc dữ liệu sử dụng tham số là kiểu object tổng quát.

```
int i = 123;
object o = i;                // implicit boxing
object o = (object) i;       // explicit boxing
int j = (int) o;             // unboxing
```

Các nhóm toán tử trong C#

Nhóm toán tử	Toán tử
Toán học	+ - * / %
Logic	& | ^ ! && || true false
Ghép chuỗi	+
Tăng, giảm	++, --
Dịch bit	<< >>
Quan hệ	== != < > <= >=
Gán	= += -= *= /= %= &= |= ^= <<= >>=
Chỉ số	[ ]
Ép kiểu	( )
Indirection và Address	* -> [ ] &

Thứ tự ưu tiên của toán tử

Nhóm toán tử	Toán tử
Primary	{x} x.y f(x) a[x] x++ x--
Unary	+ - ! ~ ++x -x (T)x
Nhân	* / %
Cộng	+ -
Dịch bit	<< >>
Quan hệ	< > <= >= is
Bằng	== !=
Logic trên bit AND	&
XOR	^
OR	|
Điều kiện AND	&&
Điều kiện OR	||
Điều kiện	?:
Assignment	= *= /= %= += -= <<= >>= &= ^= |=

Kiểu mảng

- Mảng là 1 tập các điểm dữ liệu (của cùng kiểu cơ sở), được truy cập dùng 1 số chỉ mục
- Các mảng trong C# phát sinh từ lớp cơ sở System.Array
- Mảng có thể chứa bất cứ kiểu nào mà C# định nghĩa, bao gồm các mảng đối tượng, các giao diện, hoặc các cấu trúc
- Ví dụ:

```
int [] a;
```

Kiểu mảng

Khai báo biến mảng có hai cách như sau:

1) Khai báo và khởi tạo mảng

int[] yourarr = new int[ptu];

2) Khai báo sau đó khởi tạo mảng

int[] myarr;

myarr = new int[ptu];

Khai báo mảng với số phần tử cho trước và khởi tạo giá trị cho các phần tử của mảng:

int[] me = {1,2,3,4,5};

float[] arr = {3.14f, 2.17f, 100};

float[] arr = new float [3] { 3.14f, 2.17f, 100 };

Kiểu mảng

arr.length: số phần tử của mảng

Khai báo mảng 2 chiều:

```
int [,] Mang2chieu;
Mang2chieu = new int[3,4]
```

Khai báo mảng của mảng:

```
int [][] M = new int[2][];
M[0] = new int[4];
M[1] = new int[30];
```

Kiểu string

- Kiểu string là 1 kiểu dữ liệu tham chiếu trong C#
- System.String cung cấp các hàm tiện ích như: Concat(), CompareTo(), Copy(), Insert(), ToUpper(), ToLower(), Length, Replace(),…
- Toán tử & là cách tốc ký thay cho Concat()
- Có thể truy cập các ký tự riêng lẻ của 1 chuỗi dùng toán tử chỉ mục ([ ])
- Các toán tử == và != được định nghĩa để so sánh các giá trị của các đối tượng chuỗi, chứ không phải là bộ nhớ mà chúng tham chiếu đến

Kiểu pointer

- Kiểu pointer được khai báo với dấu * ngay sau loại dữ liệu và trước tên biến cùng với từ khoá unsafe.
- Biên dịch ứng dụng C# có sử dụng kiểu dữ liệu pointer:

```
D:\csc pointer.cs /unsafe
```

Kiểu pointer

- Không giống như hai kiểu dữ liệu value và reference, kiểu pointer không chịu sự kiểm soát của garbage collector
- Garbage collector không dùng cho kiểu dữ liệu này do chúng không biết dữ liệu mà con trỏ trỏ đến.
- Vì vậy, pointer không cho phép tham chiếu đến reference hay một struct có chứa các kiểu references và kiểu tham chiếu của pointer thuộc loại kiểu không quản lý (unmanaged-type).

Tham số

- Tham trị:
  - Tham số có giá trị không thay đổi trước và sau khi thực hiện phương thức
- Tham biến:
  - Tham số có giá trị thay đổi trước và sau khi thực hiện phương thức, có thể đi sau các từ khóa: ref, out, params
  - ref: tham số đi theo sau phải khởi tạo trước khi truyền vào phương thức
  - out: tham số không cần khởi tạo trước khi truyền vào phương thức
  - params: tham số nhận đối số mà số lượng đối số là biến, từ khóa này thường sử dụng tham số là mảng.

Từ Khóa ref

```
void MyMethod()
{
    int num1 = 7, num2 = 9;
    Swap(ref num1, ref num2);
    // num1 = 9, num2 = 7
}
void Swap(ref int x, ref int y)
{
    int temp = x; x = y; y = temp;
}
```

Từ khóa out

```
void MyMethod()
{
    int num1 = 7, num2;
    Subtraction(num1, out num2);
    // num1 = 7, num2 = 5
}
void Subtraction(int x, out int y)
{
    y = x - 2;
    // y must be assigned a value
}
```

uninitialised

Từ khóa params

```
public static void Sum( out int result, params int[] myArray)
{
     result = 0;
    for (int i = 0; i < myArray.Length; i++)
        result += myArray[i];

}
```

Luôn khai báo ở cuối danh sách tham số

```
public static int Main()
{
    int s;
    Sum(out s, 1, 2, 3);
    return 0;
}
```

3 phần tử

```
public static int Main()
{
    int s;
    Sum(out s, 1, 2, 3, 4, 5, 6);
    return 0;
}
```

6 phần tử

```
public static int Main()
{
    int s;
    int[] array = { 11, 22, 33, 44 };
    Sum(out s, array);
    return 0;
}
```

Mảng array

Từ khóa var

Thay vì sử dụng khai báo biến thông thường ta có thể sử dụng từ khóa var để khai báo biến kiểu ngầm định. Một biến được khai báo bằng từ khóa này vẫn có kiểu cụ thể, chỉ khác là kiểu này trình biên dịch xác định từ giá trị được gán.

Ví dụ:

- var int_variable = 6; //int_variable is compiled as an int
- var string_variable = "Mony"; //string_variable is compiled as a string
- var int_array = new[] { 0, 1, 2 }; //int_array is compiled as int[]

Phát biểu chọn

- Phát biểu chọn (selection statement) trong C# bao gồm các phát biểu (if, if…else…, switch…case…).

Phát biểu if

```
if (expression)
        statement
if (expression)
{
        statement1
        statement1
}
```

Phát biểu if…else…

```
if (expression)
        statement1
else
        statement2
```

Phát biểu switch…case…

- Phát biểu switch…case… là phát biểu điều khiển nhiều chọn lựa bằng cách truyền điều khiển đến phát biểu case bên trong.

```
switch (expression)
{
        case constant-expression:
                statement
                jump-statement
        [default:
                statement
                jump-statement]
}
```

Phát biểu lặp

- Phát biểu vòng lặp trong C# bao gồm do, for, foreach, while.

Vòng lặp do

```
do{
    statement
}while (expression);
```

Vòng lặp while

```
while (expression){
        statement
}
```

Phát biểu lặp

Vòng lặp for

for ([initializers]; [expression]; [iterators])

statement

```
int j = 0;
for(int k = 0; j + k != 10; j++, k++)
    Console.WriteLine("j is " + j + " k is " + k);
```

Vòng lặp foreach … in

foreach (type identifier in expression)

statement

- Vòng lặp foreach lặp lại một nhóm phát biểu cho mỗi phần tử trong mảng hay tập đối tượng.
- Phát biểu dùng để duyệt qua tất cả các phần tử trong mảng hay tập đối tượng và thực thi một tập lệnh.

Phát biểu nhảy

- Phát biểu nhảy sẽ được sử dụng khi chương trình muốn chuyển đổi điều khiển.
- Phát biểu nhảy: break, continue, default, goto, return

```
static void Main(string[] args)
{
    for (int count = 1
    {
        if (count == 5
            break;
        Console.WriteL
    }
}
```

```
static void Main(string[] args)
{
    for (int count = 1; count <= 10; count++)
    {
        if (count == 5)
            continue;
        Console.WriteLine(count + " ");
    }
}
```

Tóm tắt

Statement	Example

Local variable declaration	static void Main() {
	int a;
	int b = 2, c = 3;
	a = 1;
	Console.WriteLine(a + b + c);
	}

Local constant declaration	static void Main() {
	const float pi = 3.1415927f;
	const int r = 25;
	Console.WriteLine(pi * r * r);
	}

Expression statement	static void Main() {
	int i; i = 123;
	Console.WriteLine(i);
	i++; // tăng i lên 1
	Console.WriteLine(i);
	}

Tóm tắt

Statement	Example

if statement	static void Main(string[] args) {
	if (args.Length == 0) {
	Console.WriteLine("No arguments");
	} else { Console.WriteLine("One or more arguments"); }
	}

switch statement	static void Main(string[] args) {
	int n = args.Length;
	switch (n) {
	case 0: Console.WriteLine("No arguments");
	break;
	case 1: Console.WriteLine("One argument");
	break;
	default: Console.WriteLine("{0} arguments", n);
	break;
	}
	}

Tóm tắt

Statement	Example

while statement	static void Main(string[] args) {
	int i = 0;
	while (i < args.Length) {
	Console.WriteLine(args[i]);
	i++; }
	}

do statement	static void Main() {
	string s;
	do {
	s = Console.ReadLine();
	if (s != null) Console.WriteLine(s);
	} while (s != null);
	}

for statement	static void Main(string[] args) {
	for (int i = 0; i < args.Length; i++)
	Console.WriteLine(args[i]);
	}

Tóm tắt

Statement	Example

foreach statement	static void Main(string[] args) {
	foreach (string s in args) Console.WriteLine(s);
	}

break statement	static void Main() {
	while (true) {
	string s = Console.ReadLine();
	if (s == null) break;
	Console.WriteLine(s);
	}
	}

continue statement	static void Main(string[] args) {
	for (int i = 0; i < args.Length; i++) {
	if (args[i].StartsWith("/")) continue;
	Console.WriteLine(args[i]);
	}
	}

Tóm tắt

Statement	Example

goto statement	static void Main(string[] args) {
	int i = 0;
	goto check;
	loop: Console.WriteLine(args[i++]);
	check: if (i < args.Length) goto loop;
	}

return statement	static int Add(int a, int b) { return a + b; }
	static void Main() {
	Console.WriteLine(Add(1, 2));
	return;
	}

checked and unchecked statements	static void Main() {
	int i = int.MaxValue;
	checked { Console.WriteLine(i + 1);// Exception }
	unchecked { Console.WriteLine(i + 1);// Overflow }
	}

Sử dụng biểu thức chính qui

- Sử dụng biểu thức chính qui để kiểm tra dữ liệu nhập:
  - System.Text.RegularExpression.Regex
  - Để kiểm tra dữ liệu có hợp lệ so với biểu thức chính qui hay không ta sử dụng phương thức isMatch của lớp Regex.
- Ví dụ:

```
Regex reg = new Regex(@"^\d{6}$");
if (!reg.IsMatch(txtPin.Text))
    MessageBox.Show("Dữ liệu nhập không hợp lệ");
```

Sử dụng biểu thức chính qui

- Các siêu ký tự thường dùng:

Siêu ký tự	Ý nghĩa
.	Mọi ký tự, trừ ký tự xuống dòng (\n)
\d	Ký tự số (digit)
\D	Non-digit
\s	Ký tự whitespace
\S	Non-whitespace
\w	
\W	
^	Bắt đầu bằng một chuỗi hay dòng
$	Kết thúc một chuỗi hay dòng

Sử dụng biểu thức chính qui

- Các siêu ký tự thường dùng:

Siêu ký tự	Ý nghĩa
|	Ngăn cách các biểu thức có thể so trùng
[abc]	[xyz]
[^abc]	[^xyz]
[a-z]	[A-D], [0-9]
?	Có một hoặc không có, A?B
*	Không có hoặc có nhiều, A*B
+	Có một hoặc có nhiều, A+B
{n}, {n,}	
{n,m}	

Sử dụng biểu thức chính qui

- Một số biểu thức chính qui thông dụng:

^\d+$	Chỉ chứa các chữ số
^\d{6}$	Chứa sáu chữ số (PIN)
^\w{6,8}$	Chứa từ 6 đến 8 ký tự (password đơn giản)
^\d{4}-?\d{4}-?\d{4}$	Số thẻ tín dụng (12 chữ số, có thể có dấu -)
^[\w-]+@([\w-]+\.)+[\w-]+$	Địa chỉ email

OOP in C#

Khai báo lớp

[Thuộc tính] [Bổ từ truy cập]

class <Định danh lớp> [: Lớp cơ sở]

{

<Phần thân của lớp:

các thuộc tính

phương thức >

}

Thuộc tính truy cập public, protected, internal, protected internal, private

Thuộc tính truy cập

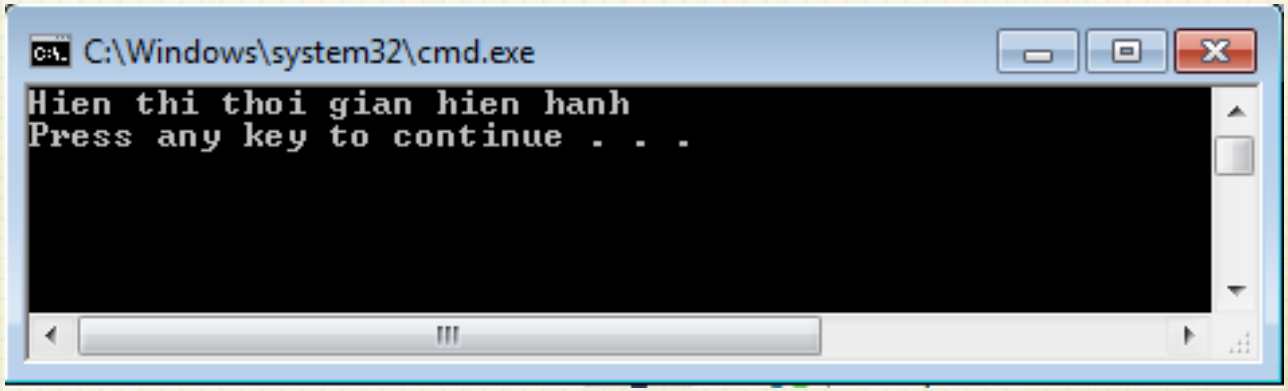
Thuộc tính	Giới hạn truy vập
public	Không hạn chế
private	Chỉ trong lớp (mặc định)
protected	Trong lớp và lớp con(lớp dẫn xuất)
internal	Trong chương trình
protected internal	Trong chương trình và trong lớp con

Ví dụ

```
01  using System;
02  public class ThoiGian
03  {
04    public void ThoiGianHienHanh()
05    {
06          Console.WriteLine("Hien thi thoi gian
            hien hanh");
07    }
08    // Các biến thành viên
09    int Nam;
10    int Thang;
11    int Ngay;
12    int Gio;
13    int Phut;
14    int Giay;
15  }
```

Ví dụ

```
16  public class Tester
17  {
18    static void Main()
19    {
20          ThoiGian t = new ThoiGian();
21          t.ThoiGianHienHanh();
22    }
23  }
```



Khởi tạo giá trị cho thuộc tính

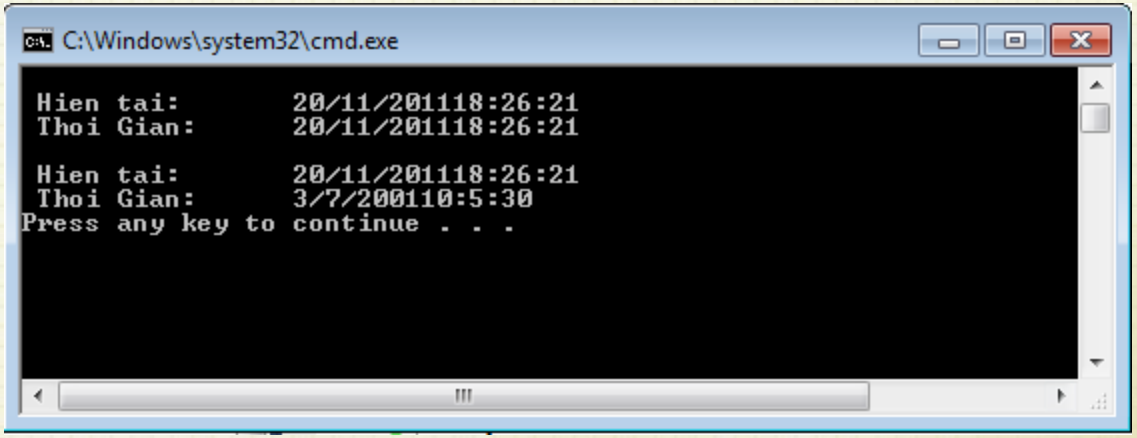
```
01  public class ThoiGian
02  {
03     public void ThoiGianHienHanh()
04     {
05            System.DateTime now = System.DateTime.Now;
06            System.Console.WriteLine("\n Hien tai: \t
                   {0}/{1}/{2}{3}:{4}:{5}",
07                 now.Day, now.Month, now.Year, now.Hour,
                   now.Minute, now.Second);
08            System.Console.WriteLine(" Thoi Gian:\t
              {0}/{1}/{2}{3}:{4}:{5}",
09                Ngay, Thang, Nam, Gio, Phut, Giay);
10     }
11     public ThoiGian (System.DateTime dt)
12     {
13            Nam = dt.Year;Thang = dt.Month;Ngay = dt.Day;
14            Gio = dt.Hour;Phut = dt.Minute;
15            Giay = dt.Second;
16     }
```

Khởi tạo giá trị cho thuộc tính

```
17      public ThoiGian(int Year, int Month, int Date, int
                Hour, int Minute)
18      {
19              Nam = Year;Thang = Month;Ngay = Date;
20              Gio = Hour;Phut = Minute;
21      }
22      private int Nam;
23      private int Thang;
24      private int Ngay;
25      private int Gio;
26      private int Phut;
27      private int Giay = 30 ; // biến được khởi tạo.
28  }
```

Khởi tạo giá trị cho thuộc tính

```
29  public class Tester
30  {
31      static void Main()
32      {
33          System.DateTime currentTime =
                                System.DateTime.Now;
34          ThoiGian t1 = new ThoiGian( currentTime );
35          t1.ThoiGianHienHanh();
36          ThoiGian t2 = new ThoiGian(2001,7,3,10,5);
37          t2.ThoiGianHienHanh();
38      }
39  }
```



Phương thức khởi tạo

Hàm tạo mặc định: giống C++

Hàm tạo có đối số: tương tự C++

```
public class MyClass
{
        public MyClass()  // zero-parameter constructor
        {
                // construction code
        }
        public MyClass(int number)   // another overload
        {
                // construction code
        }
}
```

Phương thức khởi tạo

- Lưu ý: Khai báo private cho constructor sẽ không cho phép tạo đối tượng

```
public class MyString
{
    private MyString()
    {
    }
    public static string Concat(string s1, string s2)
    {
        ...
    }
    public static int Compare(string s1, string s2)
    {
        ...
    }
}
public class Tester
{
    public static int Main()
    {

        MyString str = new MyString();
        return 0;
    }
}
```

Không thể tạo thể hiện/object

Object Initializer

Tính năng này giúp ta giảm thiểu sự dài dòng khi khai báo mới một đối tượng. Thay vì dùng cách định giá trị member thông qua instance của đối tượng, ta có thể định trực tiếp ngay khi vừa khai báo đối tượng

Ví dụ:

```
public class Person{
    public string Name { get; set; }
    public int Age { get; set; }
}
public class Program{
    static void Main(string[] args)
    {
        var person = new Person() {Name = “John", Age = 25};
    }
}
```

Named Arguments và Optional Parameters

Optional parameters: Một tham số được khai báo là tùy chọn bằng cách đặt một giá trị cho nó:

Ví dụ:

```
public MyClass(int x, int y = 5, int z = 7)
{
    .....
}
```

MyClass myClass = new MyClass(1, 2, 3); // gọi constructor của lớp MyClass

MyClass myClass = new MyClass(1, 2); // thiếu tham số z

MyClass myClass = new MyClass(1); // thiếu tham số y và z

Named Arguments và Optional Parameters

Named and optional arguments: C# không cho phép để trống các tham số ở giữa ví dụ MyClass(1, , 3). Nếu muốn bỏ trống các tham số ở giữa, mỗi tham số cần phải gán với một cái tên.

Ví dụ:

```
MyClass myClass = new MyClass(1, z: 3); // passing z by name
```

```
MyClass myClass = new MyClass(x: 1, z: 3);
```

hay

```
MyClass myClass = new MyClass(z: 3, x: 1);
```

Phương thức hủy bỏ

- C# cung cấp cơ chế thu dọn (garbage collection) và do vậy không cần phải khai báo tường minh các phương thức hủy.
- Phương thức Finalize sẽ được gọi bởi cơ chế thu dọn khi đối tượng bị hủy.
- Phương thức kết thúc chỉ giải phóng các tài nguyên mà đối tượng nắm giữ, và không tham chiếu đến các đối tượng khác.

Phương thức hủy bỏ

```
~Class1()
{
    // Thực hiện một số công việc
}

Class1.Finalize()
{
    // Thực hiện một số công việc
    base.Finalize();
}
```

Hàm hủy

```
class MyClass : IDisposable
{
  public void Dispose()
  {
    // implementation
  }
}
```

Hàm hủy

- Lớp sẽ thực thi giao diện *System.IDisposable*, tức là thực thi phương thức I*Disposable.Dispose()*.
- Không biết trước được khi nào một Destructor được gọi.
- Có thể chủ động gọi thu dọn rác bằng cách gọi phương thức *System.GC.Collect().*
- *System.GC* là một lớp cơ sở .NET mô tả bộ thu gom rác và phương thức *Collect()* dùng để gọi bộ thu gom rác.

Tham chiếu this

- Từ khóa **this** dùng để tham chiếu đến thể hiện hiện hành của một đối tượng

```
public void SetYear( int Nam)
{
   this.Nam = Nam;
}
```

- Tham chiếu this này được xem là con trỏ ẩn đến tất các phương thức **không có thuộc tính tĩnh** trong một lớp

Thành viên static

- Thành viên tĩnh được xem như một phần của lớp.
- Có thể truy cập đến thành viên tĩnh của một lớp thông qua tên lớp
- C# không cho phép truy cập đến các phương thức tĩnh và các biến thành viên tĩnh thông qua một thể hiện.
- **Không có friend**
- Phương thức tĩnh hoạt động ít nhiều giống như phương thức toàn cục

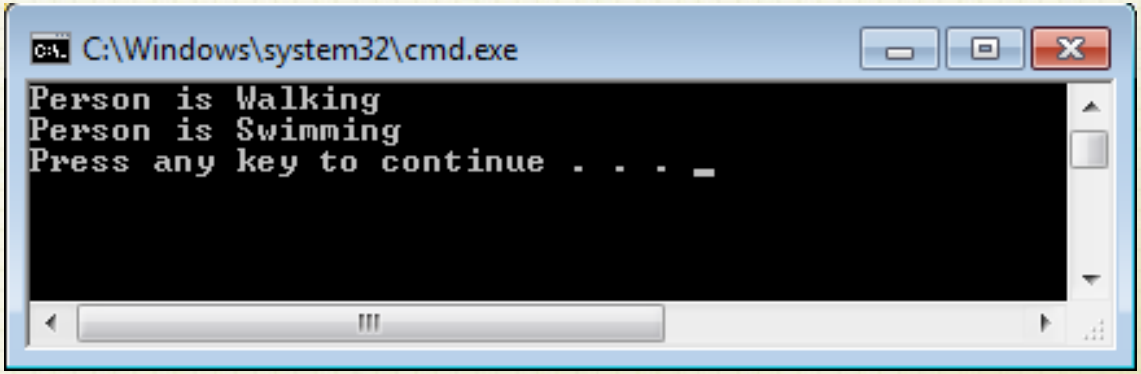
Partial Classes và Parial Methods

C# Cho phép ta dùng từ khóa **partial** để phân chia lớp thành nhiều phần có thể chứa trên nhiều file.

```
01   using System;
02
03   partial class Person
04   {
05       public void Walk()
06       {
07           Console.WriteLine("Person is Walking");
08       }
09   }
10   partial class Person
11   {
12       public void Swim()
13       {
14           Console.WriteLine("Person is Swimming");
15       }
16   }
```

Partial Classes và Parial Methods

```
17    class Program
18    {
19        static void Main(string[] args)
20        {
21            Person person = new Person();
22            person.Walk();
23            person.Swim();
24        }
25    }
```



Partial Classes và Parial Methods

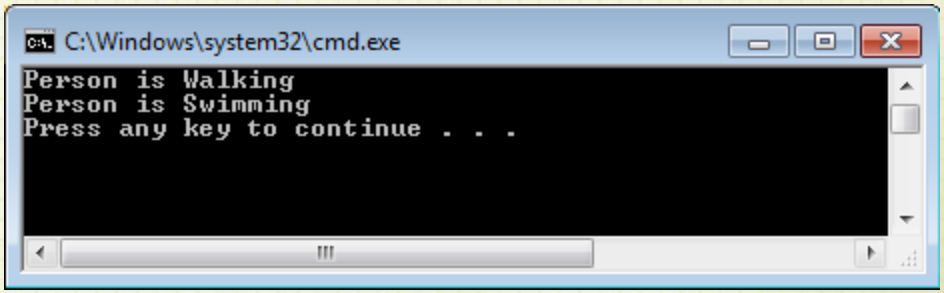
Ta cũng có thể sử dụng từ khóa partial để khai báo method nhưng ta không thể xây dựng method ở cả hai nơi của partial class

```
01   using System;
02
03   partial class Person
04   {
05       partial void Sing();
06   }
07   partial class Person
08   {
09       partial void Sing()
10       {
11           Console.WriteLine("Person is Singing");
12       }
13       public void MethodInvoker()
14       {
15           Sing();
16       }
17   }
```

Partial Classes và Parial Methods

Khi dùng partial, ta không thể dùng chung với các từ khóa public, private, protected, virual, abstract, override… Do đó nếu muốn gọi partial method thì ta phải thông qua một method khác. Trong trường hợp trên là MethodInvoker().

```
18    class Program
19    {
20        static void Main(string[] args)
21        {
22            Person person = new Person();
23            person.MethodInvoker();
24        }
25    }
```



Thuộc tính (property)

- Thuộc tính cho phép tạo ra các field read-only, write-only.
- Thuộc tính cho phép tạo ra các field "ảo" với "bên ngoài"

```
class Student{
 protected DateTime _Birthday;
 public int Age{
    get{
        return DateTime.Today().Year - _Birthday.Year;
    }
 }
}
//…
Console.Writeline("Age: {0}", chau.Age);
```

Thuộc tính (property)

- Cho phép “filter” các giá trị được ghi vào field mà không phải dùng “cơ chế” hàm set_xxx như C++.
- Bên r trong biểu th

```
class Student {
    protected DateTime _Birthday;
    public int Birthday {
        get{
            return _Birthday;
        }
        set{
            if (...)
                throw new ...
            _Birthday = value;
        }
    }
}
//...
chau.Birthday = new DateTime(2007,09,23);
Console.Writeline("Birthday: {0}", chau.Birthday);
```

Chồng hàm (overload)

- Hàm trùng tên nhau:
  - Không chấp nhận hai phương thức chỉ khác nhau về kiểu trả về.
  - Không chấp nhận hai phương thức chỉ khác nhau về đặc tính của một thông số đang được khai báo như *ref* hay *out*.

Sự kế thừa

- Một class chỉ có thể kế thừa từ một class cơ sở
- Một class có thể kế thừa từ nhiều Interface
- Đơn kế thừa:

```
class MyDerivedClass : MyBaseClass
{
        // functions and data members here
}
```

Đơn thừa kế

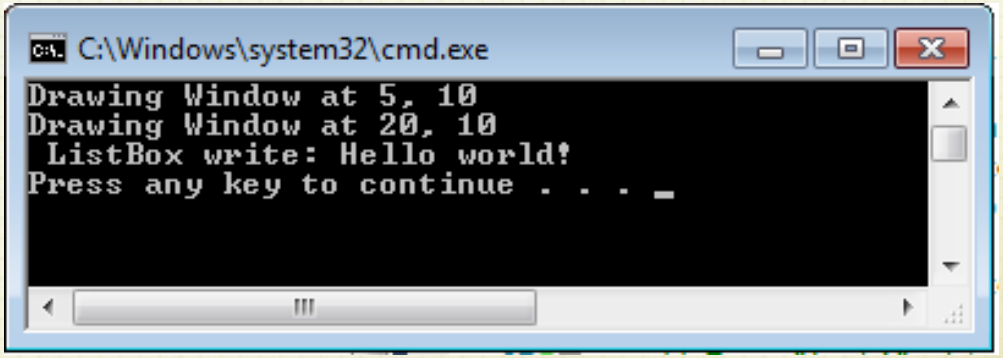
```
public class Window
{
// Hàm khởi dựng lấy hai số nguyên chỉ đến vị trí của cửa sổ trên console
        public Window( int top, int left)
        {
                this.top = top;
                this.left = left;
        }
        public void DrawWindow()   // mô phỏng vẽ cửa sổ
        {
                Console.WriteLine("Drawing Window at {0}, {1}", top, left);
        }
        // Có hai biến thành viên private do đó hai biến này sẽ không
        thấy bên trong lớp dẫn xuất.
        private int top;
        private int left;
}
```

Đơn thừa kế

```
public class ListBox: Window
{
        // Khởi dựng có tham số
        public ListBox(int top, int left,string theContents) : base(top, left)
                                                //gọi khởi dựng của lớp cơ sở
        {
                        mListBoxContents = theContents;
        }
        // Tạo một phiên bản mới cho phương thức DrawWindow
        // vì trong lớp dẫn xuất muốn thay đổi hành vi thực hiện
        // bên trong phương thức này
        public new void DrawWindow()
        {
                base.DrawWindow();
                Console.WriteLine(" ListBox write: {0}", mListBoxContents);
        }
        // biến thành viên private
        private string mListBoxContents;
}
```

Đơn thừa kế

```
public class Tester
{
        public static void Main()
        {
                // tạo đối tượng cho lớp cơ sở
                Window w = new Window(5, 10);
                w.DrawWindow();
                // tạo đối tượng cho lớp dẫn xuất
                ListBox lb = new ListBox( 20, 10, “Hello world!”);
                lb.DrawWindow();
        }
}
```



Đa hình

- Để tạo một phương thức hỗ tính đa hình: khai báo từ khóa **virtual** trong phương thức của lớp cơ sở.
- Để định nghĩa lại các hàm **virtual**, hàm tương ứng lớp dẫn xuất phải có từ khóa Override.

Phương thức virtual

- Từ khóa virtual được dùng để định nghĩa phương thức hỗ trợ đa hình.
- Đa hình nghĩa là có thể hiện thực một phương thức nhiều lần.
- Các lớp con tự lo hiện thực phiên bản của riêng mình của phương thức virtual bằng cách dùng từ khóa override.

Phương thức Override

```
class MyBaseClass
{
    public virtual string VirtualMethod()
    {
        return "This method is virtual and defined in MyBaseClass";
    }
}

class MyDerivedClass : MyBaseClass
{
    public override string VirtualMethod()
    {
        return "This method is an override defined in MyDerivedClass";
    }
}
```

Phương thức Override

Lớp Window

```
public virtual void DrawWindow()  // mô phỏng vẽ cửa sổ
{
    Console.WriteLine("Drawing Window at {0}, {1}", top, left);
}
```

Lớp Listbox

```
public override void DrawWindow()
{
    base.DrawWindow();
    Console.WriteLine(" ListBox write: {0}", mListBoxContents);
}
```

Phương thức private có thể là virtual hay override không?

Gọi các hàm của lớp cơ sở

- Cú pháp : *base.<methodname>()*

```
class CustomerAccount
{
    public virtual decimal CalculatePrice(){
        // implementation
    }
}
class GoldAccount : CustomerAccount
{
    public override decimal CalculatePrice(){
        return base.CalculatePrice() * 0.9M;
    }
}
```

Ví dụ

```
Window[] winArray = new Window[3];
winArray[0] = new Window( 1, 2 );
winArray[1] = new ListBox( 3, 4, “List box is array”);
winArray[2] = new Button( 5, 6 );

for( int i = 0; i < 3 ; i++)
{
        winArray[i].DrawWindow();
}
```

Lớp cơ sở trừu tượng

```
abstract class Building{
    public abstract decimal CalculateHeatingCost();
    // abstract method
}
```

- Một lớp abstract không được tạo thể hiện và một phương thức abstract không được thực thi mà phải được overriden trong bất kỳ lớp thừa hưởng không abstract nào.
- Nếu một lớp có phương thức abstract thì nó cũng là lớp abstract
- Một phương thức abstract sẽ tự động được khai báo *virtual*

Abstract class

```
public abstract class BankAccount {
 …
 public abstract bool IsSufficientFund(decimal Amount);
 public abstract void AddInterest();
 …
}
```

Không thể new một abstract class

Chỉ có lớp abstract mới có thể chứa abstract method

Lớp cô lập (sealed class)

- Một lớp cô lập thì không cho phép các lớp dẫn xuất từ nó.
- Để khai báo một lớp cô lập dùng từ khóa **sealed**

```
using System;
sealed class MyClass
{
        public int x;
        public int y;
}
class MainClass {
        public static void Main()
        {
                MyClass mC = new MyClass();
                mC.x = 110; mC.y = 150;
                Console.WriteLine("x = {0}, y = {1}", mC.x, mC.y);
        }
}
```

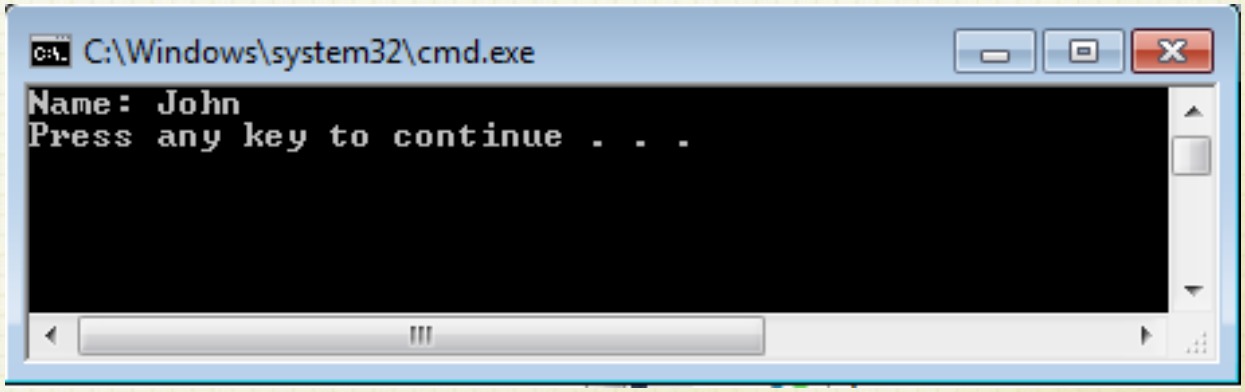
Extension Methods

Tính năng này cho phép ta thêm method vào một lớp được xây dựng sẵn mà không làm ảnh hưởng đến cấu trúc của lớp.

```
sealed class Person{//Lớp bị niêm phong, không thể kế thừa
    public string Name { get; set; }
    ……
}
static class Utility{
    //Extension method thêm vào lớp Person
    static public void ExMethod(this Person person){
        person.Name = “John";
        Console.WriteLine("Name: " + person.Name);
    }
}
```

Extension Methods

```
class Program{
    static void Main(string[] args){
        //Tạo instance của lớp Person
        var person = new Person();
        //Gọi Extension Method
         person.ExMethod();
    }
}
```



Lớp trong lớp

- Lớp được khai báo bên trong thân của lớp khác gọi là inner class/nested class, lớp ngoài gọi là outer class.
- Lớp nội có thể truy cập tất cả thành viên của lớp ngoài, kể cả private.
- Lớp nội nằm trong lớp ngoài nên nó có thể là private với lớp ngoài.
- Khi lớp nội khai báo là public thì có thể truy xuất thông qua tên của lớp ngoài: outer_class.inner_class

Lớp trong lớp

```
public class Fraction{
   private int numerator;
   private int denominator;
   public Fraction(int numerator, int denominator){
      this.numerator = numerator;
      this.denominator = denominator;
   }
   public override string ToString() {
         string str = numerator.ToString() + "/" + denominator.ToString();
         return  s;
   }
   public class FractionArtist
   {
      public void Draw(Fraction f){
         Console.WriteLine("Drawing the numerator {0}",    f.numerator);
         Console.WriteLine("Drawing the denominator {0}",  f.denominator);
      }
   }
}
```

Truy xuất được thành phần private của lớp outer

Lớp nested class

Lớp trong lớp

```
class Tester
{
    static void Main()
    {
        Fraction f1 = new Fraction(3, 4);
        Console.WriteLine("f1: {0}", f1.ToString());
        Fraction.FractionArtist  fa = new Fraction.FractionArtist();
        fa.Draw(f1);
    }
}
```

Truy xuất lớp inner qua lớp outer, lớp outer tương tự như manespace

Overload Operator

- Overload == thì phải overload !=
- Overload > thì phải overload <
- Overload >= thì phải overload <=
- Phải cung cấp các phương thức thay thế cho toán tử được nạp chồng

Overload Operator

Biểu tượng	Tên phương thức thay thế
+	Add
-	Subtract
*	Multiply
/	Divide
==	Equals
>	Compare

Overload Operator

```
01  using System;
02  public class Phanso
03  {
04      public Phanso(int ts, int ms)
05      {
06          this.ts = ts;
07          this.ms = ms;
08      }
09      public Phanso(int wholeNumber)
10      {
11          ts = wholeNumber;
12          ms = 1;
13      }
14      public static implicit operator Phanso(int theInt)
15      {
16          return new Phanso(theInt);
17      }
```

Overload Operator

```
18      public static explicit operator int(Phanso thePhanso)
19      {
20          return thePhanso.ts / thePhanso.ms;
21      }
22      public static bool operator ==(Phanso lhs, Phanso rhs)
23      {
24          if (lhs.ts == rhs.ts && lhs.ms == rhs.ms)
25          {
26              return true;
27          }
28          return false;
29      }
30      public static bool operator !=(Phanso lhs, Phanso rhs)
31      {
32          return !(lhs == rhs);
33      }
```

Overload Operator

```
34      public override bool Equals(object o)
35      {
36          if (!(o is Phanso))
37          {
38              return false;
39          }
40          return this == (Phanso)o;
41      }
42      public static Phanso operator +(Phanso lhs,Phanso rhs)
43      {
44          if (lhs.ms == rhs.ms)
45          {
46              return new Phanso(lhs.ts + rhs.ts, lhs.ms);
47          }
48          int firstProduct = lhs.ts * rhs.ms;
49          int secondProduct = rhs.ts * lhs.ms;
50          return new Phanso(firstProduct + secondProduct,
                                          lhs.ms * rhs.ms);
51      }
```

Overload Operator

```
52      public override string ToString()
53      {
54          string s = ts.ToString() + "/" + ms.ToString();
55          return s;
56      }
57      private int ts;
58      private int ms;
59  }
```

Overload Operator

```
60  public class Teste
61  {
62      static void Ma
63      {
64          Phanso f1
65          Console.Wr
66          Phanso f2
67          Console.Wr
68          Phanso f3 = f1 + f2;
69          Console.WriteLine("f1 + f2 = f3:{0}", f3.ToString());
70          Phanso f4 = f3 + 5;
71          Console.WriteLine("f4 = f3 + 5:{0}", f4.ToString());
72          Phanso f6 = 5+ f3 ;
73          Console.WriteLine("f6 = 5 + f3:{0}", f6.ToString());
74          Phanso f5 = new Phanso(2, 4);
75          if (f5 == f2){
76              Console.WriteLine("f5:{0}==f2:{1}",f5.ToString(),
                                                    f2.ToString());
77          }
78      }
79  }
```

C:\Windows\system32\cmd.exe

```
f1:3/4
f2:2/4
f1 + f2 = f3:5/4
f4 = f3 + 5:25/4
f6 = 5 + f3:25/4
f5:2/4==f2:2/4
Press any key to continue . . .
```

Bài tập OOP

- Xây dựng lớp biểu diễn khái niệm **số phức** với hai thành phần dữ liệu **thực, ảo** và các hàm thành phần: nhập, xuất, định nghĩa các toán tử cộng, trừ, nhân, chia hai số phức.

Ví dụ: Cho hai số phức $A(a_1, a_2)$, $B(b_1, b_2)$

- $A + B = (a_1+b_1, a_2+b_2)$
- $A - B = (a_1-b_1, a_2-b_2)$
- $A * B = (a_1*b_1 - a_2*b_2, a_1*b_2+a_2*b_1)$
- $A / B = \left(\dfrac{a_1 * b_1 + a_2 * b_2}{b_1^2 + b_2^2}, \dfrac{b_1 * a_2 - a_1 * b_2}{b_1^2 + b_2^2} \right)$

Interface (giao diện)

- Interface là ràng buộc, giao ước đảm bảo cho các lớp hay các cấu trúc sẽ thực hiện một điều gì đó.
- Khi một lớp thực thi một giao diện, thì lớp này báo cho các thành phần client biết rằng lớp này có **hỗ trợ các phương thức, thuộc tính, sự kiện và các chỉ mục khai báo trong giao diện**.
- Giao diện chính là phần đặc tả (không bao hàm phần cài đặt cụ thể nội dung) của 1 lớp.
- Một lớp đối tượng có thể đưa ra cùng lúc nhiều giao diện để các chương trình bên ngoài truy xuất

Interface (giao diện)

- Giống mà không giống abstract class! (khó phân biệt)
  - Interface chỉ có method hoặc property, KHÔNG có field (Abstract có thể có tất cả)
  - Tất cả member của interface KHÔNG được phép cài đặt, chỉ là khai báo (Abstract class có thể có một số phương thức có cài đặt)
  - Một interface có thể kế thừa từ nhiều interface khác
- Tên các interface nên bắt đầu bằng I…
  - Ví dụ: ICollection, ISortable

Interface (giao diện)

- Cú pháp để định nghĩa một giao diện:

[thuộc tính] [bổ từ truy cập] interface <tên giao diện> [: danh sách cơ sở]

{

    <phần thân giao diện>

}

Interface (giao diện)

- Một giao diện thì không có Constructor
- Một giao diện thì không cho phép chứa các phương thức nạp chồng.
- Nó cũng không cho phép khai báo những bổ từ trên các thành phần trong khi định nghĩa một giao diện.
- Các thành phần bên trong một giao diện luôn luôn là public và không thể khai báo virtual hay static.
- Sự kế thừa interface phải đặt sau sự kế thừa lớp

Interface (giao diện)

- Khi một class đã khai báo là implement một interface, nó phải implement tất cả method hoặc thuộc tính của interface đó
- Nếu hai interface có trùng tên method hoặc property, trong class phải chỉ rõ (explicit interface)
- Ví dụ: IMovable và IEngine đều có thuộc tính MaxSpeed

```
class ToyotaCar: Car, IMovable, IEngine {
    public IMovable.MaxSpeed {
    }
    public IEngine.MaxSpeed {
    }
}
```

Interface (giao diện)

- Thực thi nhiều giao diện:

```
public class Document : IStorable, ICompressible
```

- Mở rộng giao diện

```
interface ILoggedCompressible : ICompressible
{
    void LogSavedBytes();
}
```

- Kết hợp các giao diện:

```
interface IStorableCompressible : IStoreable, ILoggedCompressible
{
    void LogOriginalSize();
}
```

Interface vs Abstract

- Abstract: phản ánh tất cả đặc điểm (kể cả cấu trúc nội tại) chung nhất của một tập đối tượng nào đó.
- Interface: phản ánh một phần đặc điểm (bên ngoài) của một loại đối tượng. Hai đối tượng về mặt bản chất có thể rất khác nhau nhưng vẫn có thể có chung một phần đặc điểm nào đó giống nhau.
- Ví dụ: xe hơi Toyota và học sinh đều có tính chất chung là di chuyển được

Interface vs Abstract

- Một class chỉ được thừa kế từ một lớp cơ sở.
- Nhưng được phép implement (cài đặt, hiện thực hóa) nhiều loại interface khác nhau.

```
public class ToyotaCar: Car, IMovable, IUsePower
{
   …
}
public class Student: People, IMovable, IBreathable
{
   …
}
```

Xử lý lỗi

- Chương trình nào cũng có khả năng gặp phải các tình huống không mong muốn
  - Người dùng nhập dữ liệu không hợp lệ
  - Đĩa cứng bị đầy
  - File cần mở bị khóa
  - Đối số cho hàm không hợp lệ
- Xử lý như thế nào?
  - Một chương trình không quan trọng có thể dừng lại
  - Chương trình điều khiển không lưu? điều khiển máy bay?

Xử lý lỗi truyền thống

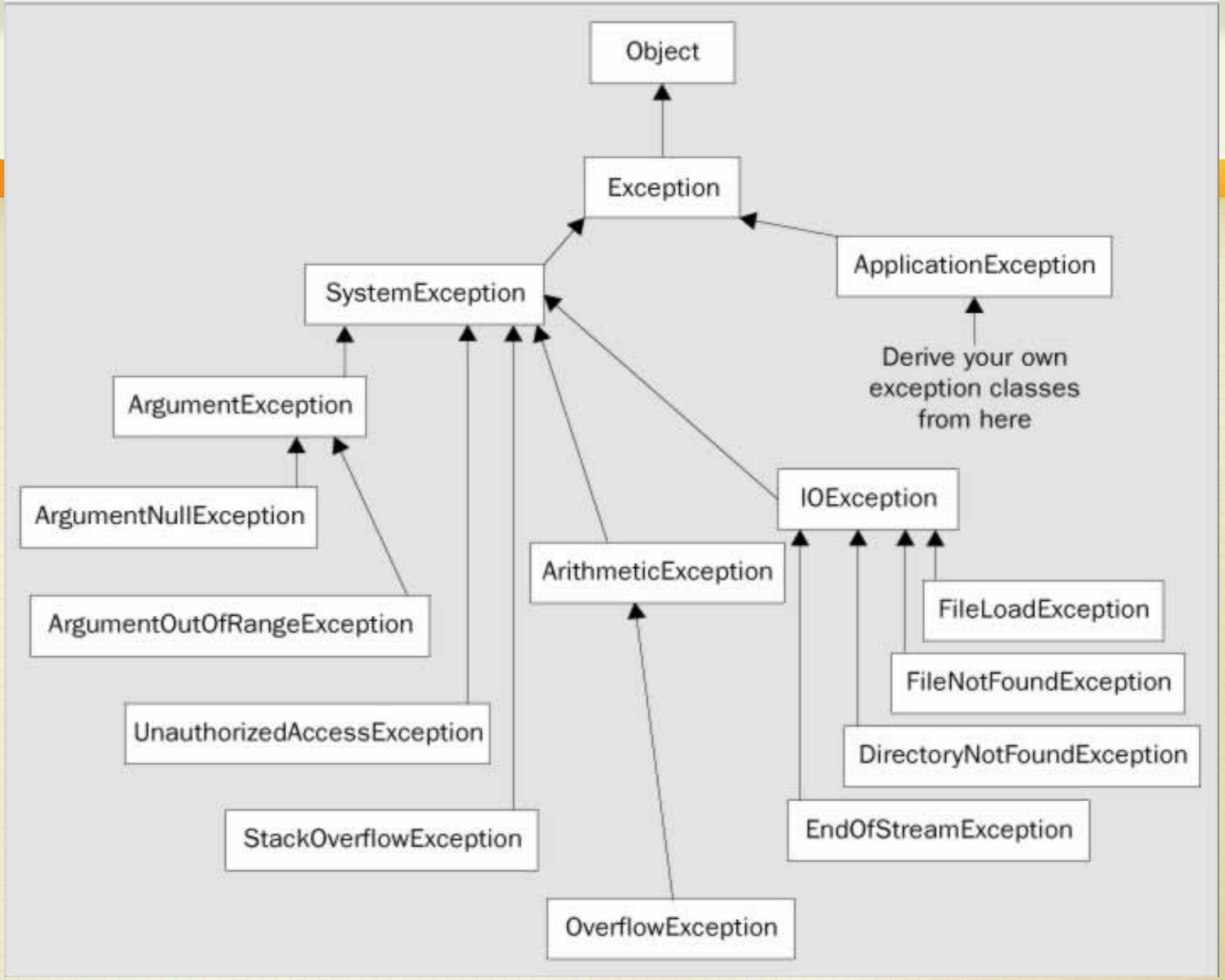
- Xử lý lỗi truyền thống thường là mỗi hàm lại thông báo trạng thái thành công/thất bại qua một mã lỗi
  - Biến toàn cục
  - Giá trị trả về
    - **int remove (const char * *filename*);**
  - Tham số phụ là tham chiếu
    - double MyDivide(double *numerator*,
      double *denominator*, **int& *status*);**

Exception

- Exception – Ngoại lệ là cơ chế thông báo và xử lý lỗi giải quyết được các vấn đề kể trên.
- Tách được phần xử lý lỗi ra khỏi phần thuật toán chính
- Cho phép 1 hàm thông báo về nhiều loại ngoại lệ
- Không thể bỏ qua ngoại lệ, nếu không, chương trình sẽ kết thúc
- Tóm lại, cơ chế ngoại lệ mềm dẻo hơn kiểu xử lý lỗi truyền thống

Xử lý ngoại lệ

- C# cho phép xử lý những lỗi và các điều kiện không bình thường với những ngoại lệ.
- Ngoại lệ là một đối tượng đóng gói những thông tin về sự cố của một chương trình không bình thường
- Khi một chương trình gặp một tình huống ngoại lệ →tạo một ngoại lệ. Khi một ngoại lệ được tạo ra, việc thực thi của các chức năng hiện hành sẽ bị treo cho đến khi nào việc xử lý ngoại lệ tương ứng được tìm thấy
- Một trình xử lý ngoại lệ là một khối lệnh chương trình được thiết kế xử lý các ngoại lệ mà chương trình phát sinh


Object
Exception
ApplicationException
SystemException
Derive your own exception classes from here
ArgumentException
ArgumentNullException
IOException
ArithmeticException
ArgumentOutOfRangeException
FileLoadException
FileNotFoundException
UnauthorizedAccessException
DirectoryNotFoundException
StackOverflowException
EndOfStreamException
OverflowException

Phát biểu throw

- Phát biểu throw dùng để phát ra tín hiệu của sự cố bất thường trong khi chương trình thực thi với cú pháp:

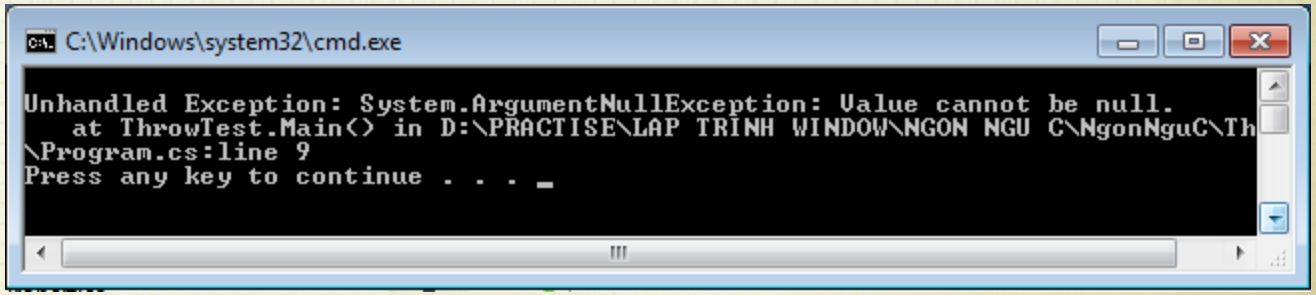
  throw [expression];

- Câu lệnh ném ngoại lệ gồm 2 phần
  - Từ khóa throw
  - Đối tượng được ném
- Ví dụ:

```
throw new System.ArgumentOutOfRangeException("Bad month");
```

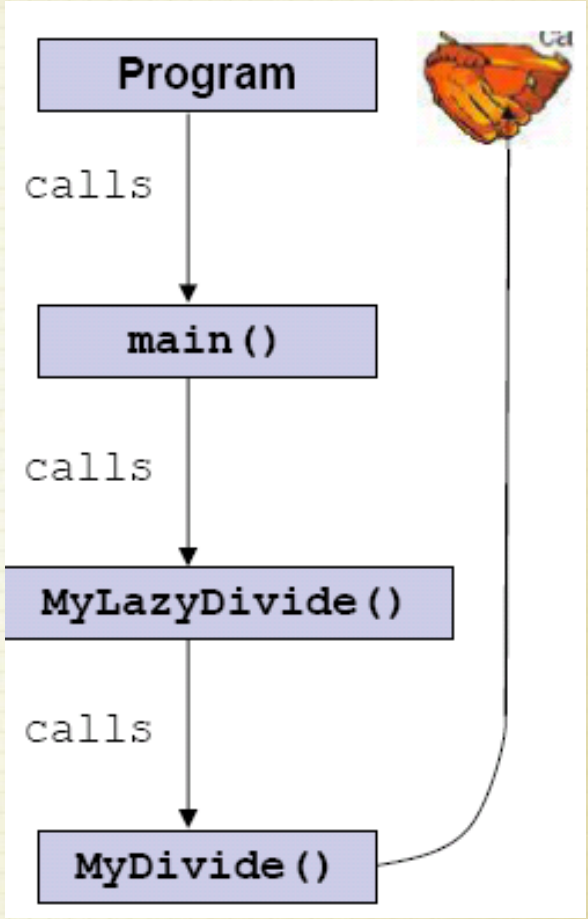
Phát biểu throw

```
01  using System;
02  public class ThrowTest
03  {
04      public static void Main()
05      {
06          string s = null;
07          if (s == null)
08          {
09              throw (new ArgumentNullException());
10          }
11          Console.Write("The string s is null");
12          // not executed
13      }
14  }
```



Phát biểu try catch

- Trong C#, một trình xử lý ngoại lệ hay một đoạn chương trình xử lý các ngoại lệ được gọi là một khối catch và được tạo ra với từ khóa catch..
- Ví dụ: câu lệnh throw được thực thi bên trong khối try, và một khối catch được sử dụng để công bố rằng một lỗi đã được xử lý



Phát biểu try catch

- Cú pháp:

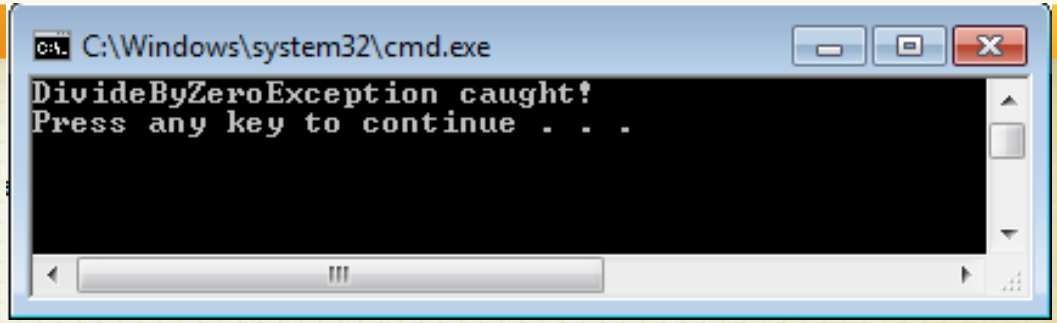
```
try{
    //Code có khả năng phát sinh ngoại lệ
}catch(Exception)
{
    //Code Xử lý lỗi
}
```

Ví dụ

```
01  using System;
02  class Test
03  {
04      static void Main(string[] args)
05      {
06          Test t = new Test();
07          t.TestFunc();
08      }
09      public double DoDivide(double a, double b)
10      {
11          if (b == 0)
12              throw new System.DivideByZeroException();
13          if (a == 0)
14              throw new System.ArithmeticException();
15          return a / b;
16      }
```

Ví dụ

```
17      public void TestFunc()
18      {
19          try
20          {
21              double a = 5;
22              double b = 0;
23              Console.WriteLine("{0} / {1} = {2}", a, b,
                                                DoDivide(a, b));
24          }
25          catch (System.DivideByZeroException)
26          {
27              Console.WriteLine("DivideByZeroException caught!");
28          }
29          catch (System.ArithmeticException)
30          {
31              Console.WriteLine("ArithmeticException caught!");
32          }
33          catch
34          {
35              Console.WriteLine("Unknown exception caught");
36          }
37      }
38  }
```



Câu lệnh finally

- Đoạn chương trình bên trong khối finally được đảm bảo thực thi mà không quan tâm đến việc khi nào thì một ngoại lệ được phát sinh

```
try
        try-block
catch
        catch-block
finally
        finally-block
```

Câu lệnh finally

1. Dòng thực thi bước vào khối try.
2. Nếu không có lỗi xuất hiện,

   - Tiến hành một cách bình thường xuyên suốt khối try, và khi đến cuối khối try, dòng thực thi sẽ nhảy đến khối finally (bước 5),

   - Nếu một lỗi xuất hiện trong khối try, thực thi sẽ nhảy đến khối catch (bước tiếp theo)
3. Trạng thái lỗi được xử lí trong khối catch
4. Vào cuối của khối catch, việc thực thi được chuyển một cách tự động đến khối finally
5. Khối finally được thực thi

Dynamic Binding

- Với dynamic binding, khi nhận được một đối tượng, chúng ta không cần phải quan tâm kiểu của đối tượng đó. Môi trường thực thi sẽ quyết định phương thức hay phép toán nào sẽ áp dụng cho đối tượng nào. Điều này tạo tạo sự linh hoạt và đơn giản khi code

Ví dụ:

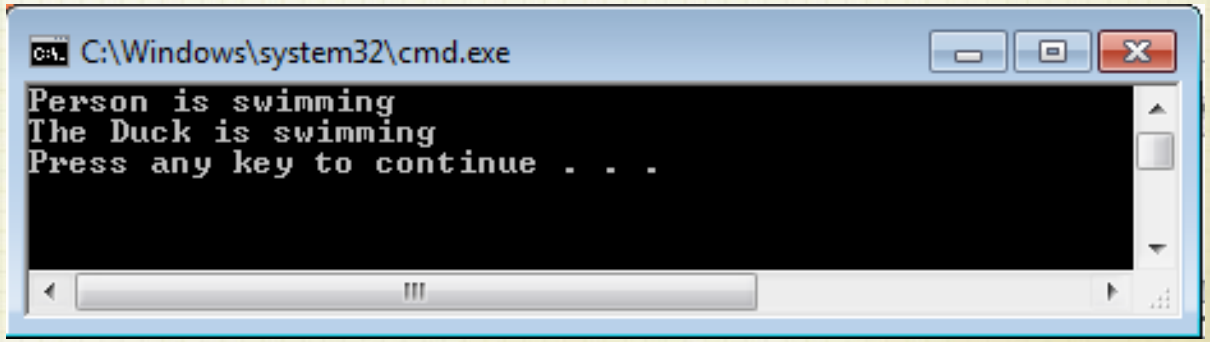
```
using System;
class Person
{
    public string Name { get; set; }
    public void Swim()
    {
        Console.WriteLine("Person is swimming");
    }
}
```

Dynamic Binding

```
10  class Duck
11  {
12      public string Weight { get; set; }
13      public void Swim()
14      {
15          Console.WriteLine("The Duck is swimming");
16      }
17  }
18  class Program
19  {
20      //Hàm có paramater là một object dynamic
21      static void InvokeSwim(dynamic obj)
22      {
23          obj.Swim();
24      }
```

Dynamic Binding

```
25  static void Main(string[] args)
26  {
27      //Khai báo 2 object dynamic khác nhau
28      dynamic person = new Person();
29      dynamic duck = new Duck();
30      //Truyền 2 object khác kiểu vào cùng 1 hàm
31      InvokeSwim(person);
32      InvokeSwim(duck);
33  }
34 }
```



Dynamic Binding

Dynamic hỗ trợ tốt các Operator +, -, *, /

```
static dynamic Sum(dynamic obj1, dynamic obj2)
{
    return obj1 + obj2;
}
static void Main(string[] args)
{
    Console.WriteLine(Sum(5, 10));
    Console.WriteLine(Sum(5.2, 10.2));
}
```

Delegates

- Trong lập trình đôi lúc chúng ta gặp trường hợp phải thực thi một hành động nào đó nhưng không biết phải gọi phương thức nào của đối tượng nào.
  - Ví dụ như khi một button được nhấn thì phải thông báo cho một đối tượng nào đó. Tuy nhiên, đối tượng đó không thể được tiên đoán trước trong lúc cài đặt lớp button. Vì vậy ta sẽ kết nối lớp button này với một đối tượng ủy thác và ủy thác cho đối tượng này trách nhiệm thực thi khi button được nhấn.

Delegates

- Tương tự như khi ta xây dựng phương thức sắp xếp trong C++.
  - Khi có input hàm sắp xếp sẽ sử dụng con trỏ hàm để gọi đến hàm compare để thực hiện sort các input.
- Về mặt bản chất Delegate là một lớp, nó có khởi tạo đối tượng và các thao tác giống như một đối tượng.
- Về mặt ý nghĩa Delegate giống như khái niệm con trỏ hàm trong C/C++.

Delegates

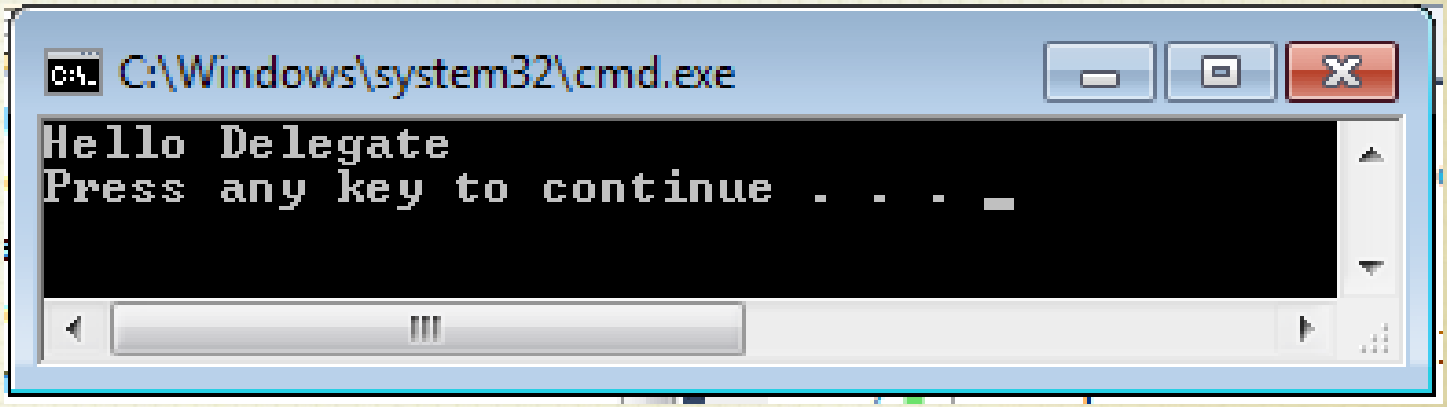
- Một delegate được khởi tạo bằng từ khóa delegate, sau đó đến kiểu trả về, tên delegate và các tham số mà delegate đó nhận vào.

Ví dụ:

```
01  using System;
02  delegate void MethodDelegate();
03  class Person
04  {
05      public void Hello()
06      {
07          Console.WriteLine("Hello Delegate");
08      }
09  }
```

Delegates

```
10  class Program
11  {
12      static void Main(string[] args)
13      {
14          Person person = new Person();
15          MethodDelegate helloDelegate =      new
                MethodDelegate(person.Hello);
16          helloDelegate(); //gọi hàm
17      }
18  }
```



Định nghĩa delegate

```
public delegate void MyDelegate1(int x, int y)
```

Delegate cho dạng hàm: void Method (int, int)

```
public delegate string MyDelegate2(float f)
```

Delegate cho dạng hàm: string Method (float)

Instance delegate

```
public void Method1 (int x, int y)
{
        //…
}
//…
MyDelegate1  del1 = new MyDelegate1(Method1);

public string Method2 (float f)
{
        //…
}
//…
MyDelegate2 del2 = new MyDelegate2(Method2);
```

Call Delegate

Gọi del1

```
int x = 5, y = 10;
del1(x, y);
```

```
del1(10, 20);
```

```
int y = 2;
del1(100, y);
```

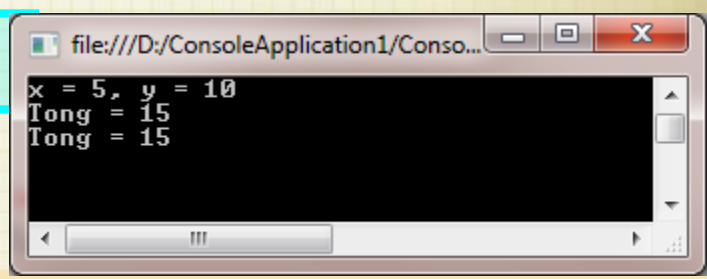
Gọi del2

```
float f =0.5f;
string s;
s = del2(f);
```

```
string s = del2(100f);
```

Multicast delegate

```
void Print(int x, int y) {
        Console.WriteLine("x = {0}, y = {1}", x, y);
}
void Sum(int x, int y) {
        Console.WriteLine("Tong = {0}", x+y);
}
MyDelegate1 mulDel = new MyDelegate1(Print);
mulDel += new MyDelegate1(Sum);
mulDel(5, 10);
mulDel -= new MyDelegate1(Print);
mulDel(5,10);
```



Ứng dụng delegate

- Như vậy, ta có thể nói delegate dùng để định nghĩa một nguyên mẫu hàm.
- Tại sao không dùng interface?
  - C# cần sự đơn giản và nhanh chóng
- Tại sao không dùng con trỏ hàm của C/C++
  - Con trỏ hàm không an toàn

Ứng dụng delegate

- Định nghĩa nguyên mẫu hàm để làm gì?
  - Chủ yếu để hình thành cơ chế truyền dữ liệu qua sự kiện.
  - Khi đối tượng A phát sinh một sự kiện, nó muốn truyền sự kiện này sang B. Nhưng A được viết trước (A là một button chẳng hạn) được viết trước B (B là MyForm chẳng hạn) nên A không thể biết B có những phương thức nào để gọi. Nên cần định nghĩa trước một nguyên mẫu cho A rồi implement trong B, rồi A gọi B qua nguyên mẫu này.

Event

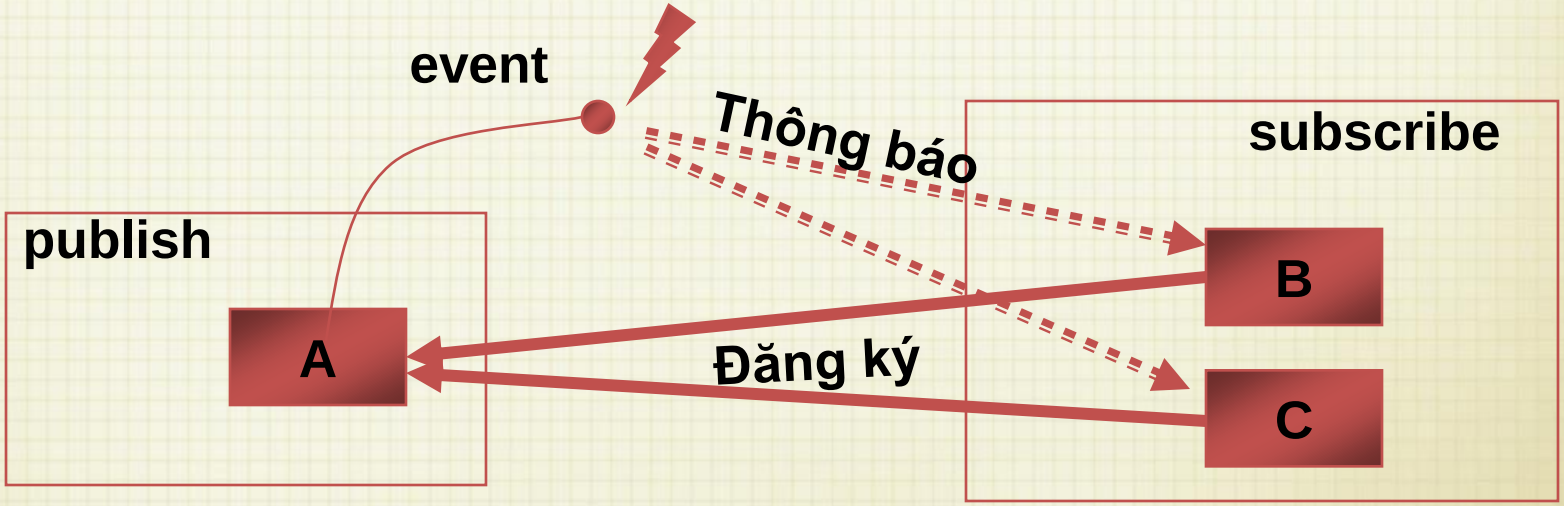
- Trong môi trường giao diện đồ họa, đòi hỏi chương trình phải đáp ứng các sự kiện.
  - Sự kiện là một hành động nào đó xảy ra và ta phải đáp ứng lại sự kiện đó.
  - Cơ chế thông điệp giữa các lớp hay các đối tượng.
  - Có thể thông báo cho lớp khác biết được khi một lớp có phát sinh điều gì đó.
  - Publisher: Lớp phát sinh sự kiện.
  - Subscriber: Lớp nhận hay xử lý khi sự kiện xảy ra.

Event

- Trong môi trường giao diện GUIs (Graphical User Interfaces):
  - Button đưa ra sự kiện “Click”, cho phép lớp khác có thể đáp ứng (xử lý) khi sự kiện này xảy ra.
  - Ví dụ: Button “Add” trong Form, khi sự kiện click xảy ra thì Form thực hiện lấy dữ liệu từ các TextBox đưa vào ListBox…

Publishing & Subscribing

- Một lớp có publish một tập các event cho phép các lớp khác subscribe
  - Button là lớp publish đưa ra event: click
  - Form là lớp subscribe có phần xử lý riêng khi *"click"* của Button kích hoạt.



Event & Delegate

- Sự kiện trong C# được thực thi nhờ uỷ thác
  - Lớp publishing định nghĩa ủy thác
  - Những lớp subscribing phải thực thi
  - Khi sự kiện xuất hiện thì phương thức của lớp subscribing được gọi thông qua uỷ thác.
- Phương thức để xử lý sự kiện gọi là trình xử lý sự kiện (event handler)

Event & Delegate

- Trình xử lý sự kiện trong .NET Framework được mô tả như sau:
  - Trả về giá trị void
  - Tham số 1: Nguồn phát sinh sự kiện, đây chính là đối tượng publisher.
  - Tham số 2: Là đối tượng thuộc lớp dẫn xuất từ EventArgs.
- EventArgs là lớp cơ sở cho tất cả các dữ liệu về sự kiện.

Event & Delegate

- Khai báo delegate xử lý sự kiện

```
public delegate void HandlerName(object obj, EventArgs arg);
```

- Khai báo event

```
public event  HandlerName OnEventName;
```

- Các lớp muốn xử lý khi sự kiện OnEventName phát sinh thì phải thực thi event handler

Event & Delegate – Ví dụ

- Xây dựng 1 lớp thực hiện yêu cầu: "cứ mỗi giây sẽ phát sinh 1 sự kiện"
- Cho phép 2 lớp khác đăng ký xử lý sự kiện này, mỗi lớp có cách xử lý riêng:
  - Lớp A: Hiển thị thời gian theo "mô phỏng đồng hồ analog"
  - Lớp B: Hiển thị thời gian theo "mô phỏng đồng hồ digital"

Event & Delegate – Ví dụ

- Tạo một lớp Clock:
  - Khai báo một event: OnSecondChange
  - Một phương thức Run: Cứ một giây thì phát sinh sự kiện OnSecondChange
- Tạo 2 lớp:
  - AnalogClock và DigitalClock nhận xử lý sự kiện OnSecondChange của lớp Clock

Event & Delegate – Ví dụ

- Khai báo delegate xử lý event

Tên delegate xử lý sự kiện

```
delegate void SecondChangeHandler(object clock, EventArgs info);
```

Đối tượng phát sinh event

Tham số kiểu EventArgs

Kiểu delegate

Tên của event

- Khai báo event có hàm xử lý mô tả trên

```
event SecondChangeHandler OnSecondChange;
```

Từ khóa event: Thể hiện cơ chế publishing & subscribing

Event & Delegate – Ví dụ

- Kích hoạt sự kiện

Kiểm tra xem có hàm xử lý được đăng ký hay không?

```
if (OnSecondChange != null)
        OnSecondChange(this, new EventArgs());
```

Gọi hàm xử lý sự kiện đã đăng ký

Event & Delegate – Ví dụ

- Lớp Clock:
  - Khai báo và phát sinh sự kiện OnSecondChange

```
public class Clock{
    public delegate void
        SecondChangeHandler(object clock, EventArgs info);

    public event SecondChangeHandler OnSecondChange;

    public void Run()
    {
        while (true){
            Thread.Sleep(1000);
            if (OnSecondChange != null)
                OnSecondChange(this, new EventArgs());
        }
    }
}
```

Event & Delegate – Ví dụ

- Lớp DigitalClock
  - Định nghĩa trình xử lý sự kiện của Clock
    - Đúng mô tả delegate hàm xử lý của lớp Clock
    - Thực hiện một số thao tác riêng của DigitalClock
  - Đăng ký xử lý với trình xử lý sự kiện trên khi có sự kiện OnSecondChange của Clock
    - Chức năng đăng ký với lớp Clock là có xử lý khi sự kiện OnSencondChange của Clock phát sinh
    - Ủy thác cho lớp Clock sẽ gọi trình xử lý định nghĩa bên trên của DigitalClock

Event & Delegate – Ví dụ

- Trình xử lý của DigitalClock

Tên của trình xử lý Đối tượng phát sinh sự kiện

```
public void Show(object obj, EventArgs args)
{
        DateTime date = DateTime.Now;
        Console.WriteLine("Digital Clock: {0}:{1}:{2}",
          date.Hour, date.Minute, date.Second);
}
```

Event & Delegate – Ví dụ

- Đăng ký xử lý sự kiện

Đối tượng này sẽ phát sinh sự kiện

```
public void Subscribe(Clock theClock)
{
        theClock.OnSecondChange +=
            new Clock.SecondChangeHandler(Show);
 }
```

Ủy thác phương thức Show cho OnSecondChange

Event & Delegate – Ví dụ

- Lớp DigitalClock: Nhận xử lý sự kiện OnSecondChange của lớp Clock

```
public class DigitalClock
{
    public void Subscribe(Clock theClock)
    {
        theClock.OnSecondChange +=
                    new Clock.SecondChangeHandler(Show);
    }
    public void Show(object obj, EventArgs args)
    {
        DateTime date = DateTime.Now;
        Console.WriteLine("Digital Clock: {0}:{1}:{2}",
                    date.Hour, date.Minute, date.Second);
    }
}
```

Event & Delegate – Ví dụ

- Lớp AnalogClock
  - Định nghĩa trình xử lý sự kiện của Clock
    - Đúng mô tả delegate hàm xử lý của lớp Clock
    - Thực hiện một số thao tác riêng của AnalogClock
  - Đăng ký xử lý với trình xử lý sự kiện trên khi có sự kiện OnSecondChange của Clock
    - Chức năng đăng ký với lớp Clock là có xử lý khi sự kiện OnSencondChange của Clock phát sinh
    - Ủy thác cho lớp Clock sẽ gọi trình xử lý định nghĩa bên trên của AnalogClock

Event & Delegate – Ví dụ

- Lớp AnalogClock

```
public class AnalogClock
{
    public void Subscribe(Clock theClock)
    {
        theClock.OnSecondChange +=
                    new Clock.SecondChangeHandler(Show);
    }
    public void Show(object obj, EventArgs args)
    {
        DateTime date = DateTime.Now;
        Console.WriteLine("Analog Clock: {0}:{1}:{2}",
                    date.Hour, date.Minute, date.Second);
    }
}
```

Event & Delegate – Ví dụ

- Minh họa cơ chế event

```
static void Main(string[] args)
{
    Clock myClock = new Clock();
    AnalogClock c1 = new AnalogClock();
    DigitalClock c2 = new DigitalClock();
    c1.Subscribe(myClock);
    c2.Subscribe(myClock);
    myClock.Run();

}
```

Phát sinh sự kiện

Đăng ký xử lý sự kiện của đối tượng myClock!

Event & Delegate – Ví dụ

- Minh họa cơ chế event

```
static void Main(string[] args)
{
    Clock myClock = new Clock();
    AnalogClock c1 = new AnalogClock();
    DigitalClock c2 = new DigitalClock();

    myClock.OnSecondChange += new
    Clock.SecondChangeHandler(c1.Show);
    myClock.OnSecondChange += new
    Clock.SecondChangeHandler(c2.Show);

    myClock.Run();
}
```

Nhận xét ví dụ Event & Delegate

- Hạn chế:
  - Mỗi lớp subscibing phải lấy giờ hiện hành riêng ➔trùng
  - Nên truyền tham số từ lớp publishing ngay khi phát sinh sự kiện
- Giải pháp:
  - Khi phát sinh sự kiện, truyền thời gian hiện hành ➔lớp subscribing sẽ sử dụng tham số này
  - Tạo một lớp TimeEventArgs kế thừa từ EventArgs: Chứa thời gian sẽ truyền đi

Minh họa EventArgs

- Tạo lớp chứa tham số truyền cho trình xử lý sự kiện
  - Lớp dẫn xuất từ EventArgs
  - Chứa các thông tin về: giờ, phút, giây
- Bắt buộc phải dẫn xuất từ EventArgs
  - Do mô tả của tr
    là lớp dẫn xuất

```
public class TimeEventArgs : EventArgs{
    public readonly int Second;
    public readonly int Minute;
    public readonly int Hour;
    public TimeEventArgs(int s, int m, int h){
        Second = s;
        Minute = m;
        Hour = h;
    }
}
```

Minh họa EventArgs

- Trong lớp Clock khai báo lại trình xử lý sự kiện như sau:

```
public delegate void
 SecondChangeHandler(object obj, TimeEventArgs arg);
```

Sử dụng tham số thứ hai có kiểu TimeEventArgs

Minh họa EventArgs

- Khi kích hoạt sự kiện thì truyền tham số {giờ, phút, giây}

```
public void Run()
{
    while (true){
        Thread.Sleep(1000);
        if (OnSecondChange != null)
        {
            DateTime date = DateTime.Now;
            TimeEventArgs timeArg = new
                TimeEventArgs(date.Second, date.Minute, date.Hour);
            OnSecondChange(this, timeArg);
        }
    }
}
```

Minh họa EventArgs

- Các lớp DigitalClock và AnalogClock sử dụng tham số truyền vào

```
public class AnalogClock
{
    public void Subcribe(Clock theClock)
    {
        theClock.OnSecondChange +=
                    new Clock.SecondChangeHandler(Show);
    }
    public void Show(object obj, TimeEventArgs timeArg)
    {
        Console.WriteLine("Analog Clock: {0}:{1}:{2}",
                timeArg.Hour, timeArg.Minute, timeArg.Second);
    }
}
```

Giao diện tập hợp

Môi trường .NET cung cấp những giao diện chuẩn cho việc liệt kê, so sánh, và tạo các tập hợp.

IEnumerable	Khi một lớp cài đặt giao diện này, đối tượng thuộc lớp đó có thể dùng trong câu lệnh foreach.
ICollection	Thực thi bởi tất cả các tập hợp để cung cấp phương thức CopyTo() cũng như các thuộc tính Count, ISReadOnly, ISSynchronized, và SyncRoot.
IComparer	So sánh giữa hai đối tượng lưu giữ trong tập hợp để sắp xếp các đối tượng trong tập hợp.
IList	Sử dụng bởi những tập hợp mảng được chỉ mục
IDictionary	Dùng trong các tập hợp dựa trên key va value
IDictionaryEnumerator	Cho phép liệt kê dùng câu lệnh foreach qua tập hợp hỗ trợ IDictionary.

Interface IEnumerable

- Interface này chỉ có một phương thức duy nhất là GetEnumerator(), công việc của phương thức là trả về một sự thực thi đặc biệt của IEnumerator.
- Mục đích của interface IEnumerable là cho phép chúng ta có thể sử dụng từ khóa **foreach** trên đối tượng của class cài đặt interface này.

Interface IEnumerator

- Interface IEnumerator bao gồm 2 phương thức quan trọng là MoveNext, Reset và thuộc tính Current.
  - Thuộc tính Current trả về phần tử hiện tại đang được duyệt tới trong danh sách.
  - MoveNext dùng để đi đến phần tử tiếp theo trong danh sách (hay nói cách khác thay đổi giá trị của Property Current).
  - Reset dùng để đưa con trỏ hiện tại về vị trí ban đầu. Vị trí ban đầu này là vị trí nằm ngày trước phần tử đầu tiên trong danh sách.

Ví dụ

```
01  using System;
02  using System.Collections;
03  public class Person
04  {
05      public Person(string fName, string lName)
06      {
07          this.firstName = fName;
08          this.lastName = lName;
09      }
10      public string firstName;
11      public string lastName;
12  }
13  public class People : IEnumerable
14  {
15      private Person[] _people;
16      // Khởi tạo
17      public People(Person[] pArray)
18      {
19          _people = new Person[pArray.Length];
20
21          for (int i = 0; i < pArray.Length; i++)
22          {
23              _people[i] = pArray[i];
24          }
25      }
```

Ví dụ

```
26      IEnumerator IEnumerable.GetEnumerator()
27      {
28          return (IEnumerator)GetEnumerator();
29      }
30      public PeopleEnum GetEnumerator()
31      {
32          return new PeopleEnum(_people);
33      }
34  }
35
36  public class PeopleEnum : IEnumerator
37  {
38      public Person[] _people;
39      int position = -1;
40      // Khởi tạo
41      public PeopleEnum(Person[] list)
42      {
43          _people = list;
44      }
45      // Tăng vị trí
46      public bool MoveNext(){
47          position++;
48          return (position < _people.Length);
49  }
```

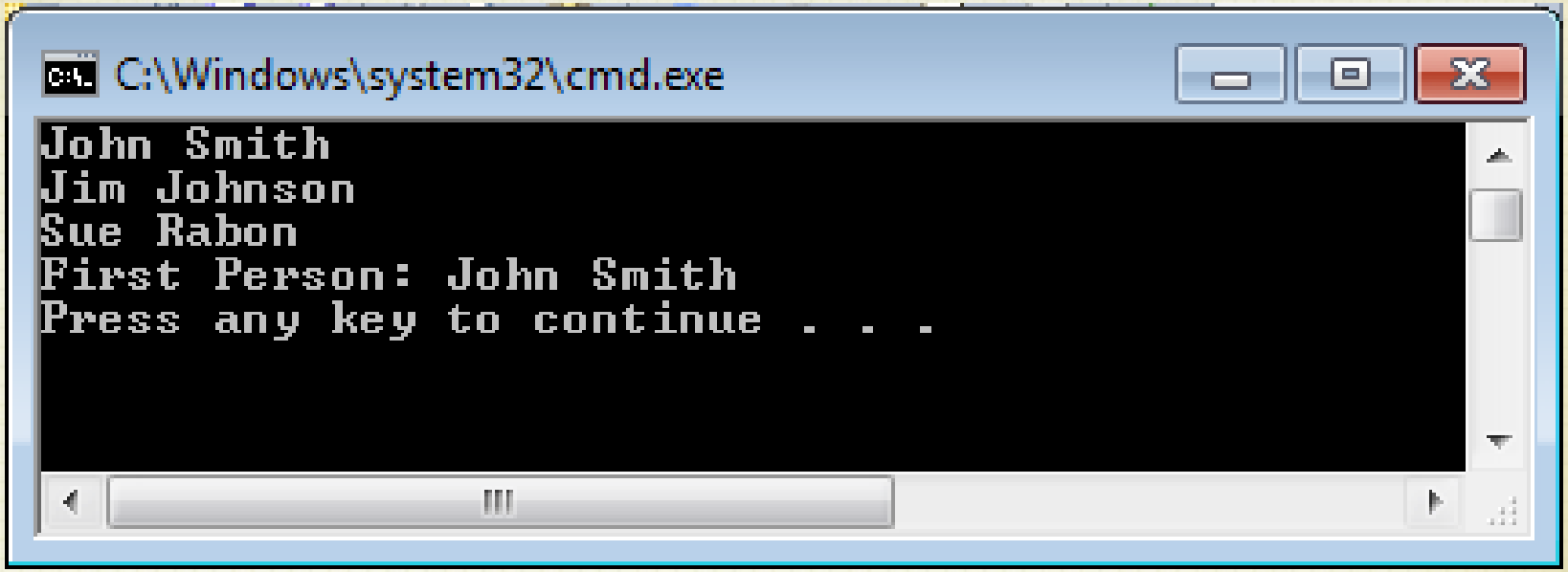
Ví dụ

```
51      public void Reset()
52      {
53          position = -1;
54      }
55      object IEnumerator.Current
56      {
57          get
58          {
59              return Current;
60          }
61      }
62      public Person Current
63      {
64          get
65          {
66              try
67              {
68                  return _people[position];
69              }
70              catch (IndexOutOfRangeException)
71              {
72                  throw new InvalidOperationException();
73              }
74          }
75      }
76  }
```

Ví dụ

```
77  class Program
78  {
79      static void Main()
80      {
81          Person[] peopleArray = new Person[3]
82          {
83              new Person("John", "Smith"),
84              new Person("Jim", "Johnson"),
85              new Person("Sue", "Rabon"),
86          };
87          People peopleList = new People(peopleArray);
88          // liệt kê danh sách dùng foreach
89          foreach (Person p in peopleList)
90              Console.WriteLine(p.firstName + " " + p.lastName);
91          PeopleEnum peopleEnum = peopleList.GetEnumerator();
92          peopleEnum.Reset();
93          // lấy người đầu tiên trong danh sách
94          peopleEnum.MoveNext();
95          Person firstPerson = peopleEnum.Current;
96          Console.WriteLine("First Person: {0} {1}",
    firstPerson.firstName , firstPerson.lastName);
97      }
98  }
```

Ví dụ


C:\Windows\system32\cmd.exe
John Smith
Jim Johnson
Sue Rabon
First Person: John Smith
Press any key to continue . . .

Interface ICollection

- Interface **ICollection** cung cấp các thuộc tính: Count, IsSynchronized, và SyncRoot.

- Ngoài ra ICollection cũng cung cấp một phương thức CopyTo(). Thuộc tính thường được sử dụng là Count, thuộc tính này trả về số thành phần trong tập hợp.

Interface IComparable

- Vai trò của **IComparable** là cung cấp một phương pháp ComparteTo() dùng để so sánh hai đối tượng.

Interface IComparable

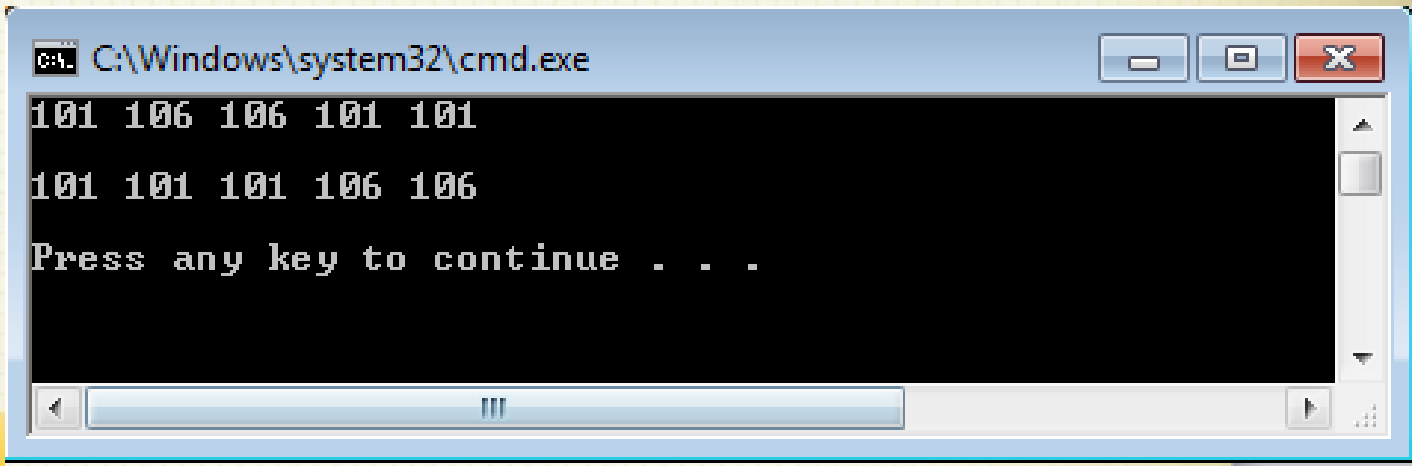
```
01  using System;
02  using System.Collections;
03  public class Employee : IComparable
04  {
05      private int empID;
06      public Employee(int empID)
07      {
08          this.empID = empID;
09      }
10      public override string ToString()
11      {
12          return empID.ToString();
13      }
14      public int EmpID
15      {
16          get
17          {
18              return empID;
19          }
20          set
21          {
22              this.empID = value;
23          }
24      }
```

Interface IComparable

```
25      public int CompareTo(Object o)
26      {
27          Employee r = (Employee)o;
28          return this.empID.CompareTo(r.empID);
29      }
30  }
31  public class Tester
32  {
33      static void Main()
34      {
35          ArrayList empArray = new ArrayList();
36          Random r = new Random();
37          for (int i = 0; i < 5; i++)
38          {
39              empArray.Add(new Employee(r.Next(10) + 100));
40          }
```

Interface IComparable

```
41          // In tất cả nội dung của mảng
42          for (int i = 0; i < empArray.Count; i++)
43          {
44              Console.Write("{0} ", empArray[i].ToString());
45          }
46          Console.WriteLine("\n");
47          // Sắp xếp lại mảng Employee dựa theo phương thức CompareTo()
48          empArray.Sort();
49          // Hiển thị tất cả nội dung của mảng Employee
50          for (int i = 0; i < empArray.Count; i++)
51          {
52              Console.Write("{0} ", empArray[i].ToString());
53          }
54          Console.WriteLine("\n");
55      }
56  }
```



Interface IComparer

- Interface **IComparer** cung cấp phương thức Compare(), để so sánh hai phần tử trong một tập hợp có thứ tự.
- Phương thức Compare() thường được thực thi bằng cách gọi phương thức CompareTo() của một trong những đối tượng.
- Nếu chúng ta muốn tạo ra những lớp có thể được sắp xếp bên trong một tập hợp thì chúng ta cần thiết phải thực thi IComparable.

Interface IComparer

```
001  using System;
002  using System.Collections;
003  public class Employee: IComparable
004  {
005      private int empID;
006      private int yearsOfSvc = 1;
007      public Employee(int empID)
008      {
009          this.empID = empID;
010      }
011      public Employee(int empID, int yearsOfSvc)
012      {
013          this.empID = empID;
014          this.yearsOfSvc = yearsOfSvc;
015      }
016      public override string ToString()
017      {
018          return "ID: " + empID.ToString() + ". Years of Svc: " +
     yearsOfSvc.ToString();
019      }
020      // Phương thức tĩnh để nhận đối tượng Comparer
021      public static EmployeeComparer GetComparer()
022      {
023          return new Employee.EmployeeComparer();
024      }
```

Interface IComparer

```
025     public int CompareTo(Object rhs)
026     {
027         Employee r = (Employee)rhs;
028         return this.empID.CompareTo(r.empID);
029     }
030     // Thực thi đặc biệt được gọi bởi custom comparer
031     public int CompareTo(Employee rhs,
    Employee.EmployeeComparer.ComparisionType which)
032     {
033         switch (which)
034         {
035             case Employee.EmployeeComparer.ComparisionType.EmpID:
036                 return this.empID.CompareTo(rhs.empID);
037             case Employee.EmployeeComparer.ComparisionType.Yrs:
038                 return this.yearsOfSvc.CompareTo(rhs.yearsOfSvc);
039         }
040         return 0;
041     }
042
043     // Lớp bên trong thực thi IComparer
044     public class EmployeeComparer : IComparer
045     {
046         private Employee.EmployeeComparer.ComparisionType
    whichComparision;
```

Interface IComparer

```
047         // Định nghĩa kiểu liệt kê
048         public enum ComparisionType
049         {
050             EmpID, Yrs
051         };
052         // Yêu cầu những đối tượng Employee tự so sánh với nhau
053         public int Compare(object lhs, object rhs)
054         {
055             Employee l = (Employee)lhs;
056             Employee r = (Employee)rhs;
057             return l.CompareTo(r, WhichComparision);
058         }
059         public Employee.EmployeeComparer.ComparisionType
    WhichComparision
060         {
061             get
062             {
063                 return whichComparision;
064             }
065             set
066             {
067                 whichComparision = value;
068             }
069         }
070     }
071 }
072
```

Interface IComparer

```
073  public class Teser
074  {
075      static void Main()
076      {
077          ArrayList empArray = new ArrayList(); Random r = new
     Random();
078          for (int i = 0; i < 5; i++)
079          {
080              empArray.Add(new Employee(r.Next(10) + 100,
     r.Next(20)));
081          }
082          // Hiển thị tất cả nội dung của mảng Employee
083          for (int i = 0; i < empArray.Count; i++)
084          {
085              Console.Write("\n{0} ", empArray[i].ToString());
086          }
087          Console.WriteLine("\n");
088          // Sắp xếp mảng theo empID
089          Employee.EmployeeComparer c = Employee.GetComparer();
090          c.WhichComparision =
     Employee.EmployeeComparer.ComparisionType.EmpID;
091          empArray.Sort(c);
```

Interface IComparer

```
092             // Hiển thị nội dung của mảng
093             for (int i = 0; i < empArray.Count; i++)
094             {
095                 Console.Write("\n{0} ", empArray[i].ToString());
096             }
097             Console.WriteLine("\n");
098             // Sắp xếp mảng theo yearsOfSvc
099             c.WhichComparision =
    Employee.EmployeeComparer.ComparisionType.Yrs;
100             empArray.Sort(c);
101             // Hiển thị nội dung của mảng
102             for (int i = 0; i < empArray.Count; i++)
103             {
104                 Console.Write("\n{0} ", empArray[i].ToString());
105             }
106             Console.WriteLine("\n");
107         }
108     }
```

Interface IComparer

C:\Windows\system32\cmd.exe

```
ID: 106. Years of Svc: 1
ID: 104. Years of Svc: 7
ID: 104. Years of Svc: 10
ID: 100. Years of Svc: 13
ID: 109. Years of Svc: 11


ID: 100. Years of Svc: 13
ID: 104. Years of Svc: 10
ID: 104. Years of Svc: 7
ID: 106. Years of Svc: 1
ID: 109. Years of Svc: 11


ID: 106. Years of Svc: 1
ID: 104. Years of Svc: 7
ID: 104. Years of Svc: 10
ID: 109. Years of Svc: 11
ID: 100. Years of Svc: 13

Press any key to continue . . .
```

Interface IDictionary

- Interface **IDictionary** là Interface đại diện chung cho kiểu tập hợp dùng cặp key và value
- **IDictionary** cung cấp một thuộc tính public là Item. Thuộc tính Item cho phép truy cập phần tử trong tập hợp thông qua toán tử chỉ mục ([]) giống như truy cập mảng.

IDictionaryEnumerator

- Những đối tượng **IDictionary** cũng hỗ trợ vòng lặp **foreach** bằng việc thực thi phương thức GetEnumerator(), phương thức này trả về một IDictionaryEnumerator.

ArrayList

- Lớp ArrayList là một kiểu dữ liệu giống như kiểu mảng nhưng kích thước của nó có thể được thay đổi động theo yêu cầu.

Thuộc tính	Mô tả
Capacity	Thuộc tính để get hay set số thành phần trong ArrayList.
Count	Thuộc tính dùng để xác định số phần tử có trong ArrayList
IsFixedSize	Thuộc tính kiểm tra xem kích thước của ArrayList có cố định hay không
IsReadOnly	Thuộc tính kiểm tra xem ArrayList có thuộc tính chỉ đọc hay không.

ArrayList

Phương thức	Mô tả
Add()	Phương thức public để thêm một đối tượng vào ArrayList
AddRange()	Phương thức public để thêm nhiều thành phần của một ICollection vào cuối của ArrayList
Clear()	Xóa tất cả các thành phần từ ArrayList
Clone()	Tạo một bản copy
Contains()	Kiểm tra một thành phần xem có chứa trong mảng hay không
CopyTo()	Phương thức public nạp chồng để sao chép một ArrayList đến một mảng một chiều.
GetEnumerator()	Phương thức public nạp chồng trả về một enumerator dùng để lặp qua mảng
Item()	Thiết lập hay truy cập thành phần trong mảng tại vị trí xác định. Đây là bộ chỉ mục cho lớp ArrayList.

ArrayList

Phương thức	Mô tả
IndexOf()	Phương thức public nạp chồng trả về chỉ mục vị trí đầu tiên xuất hiện giá trị
Insert()	Chèn một thành phần vào trong ArrayList
InsertRange(0	Chèn một dãy tập hợp vào trong ArrayList
LastIndexOf()	Phương thức public nạp chồng trả về chỉ mục trị trí cuối cùng xuất hiện giá trị.
Remove()	Xóa sự xuất hiện đầu tiên của một đối tượng xác định.
RemoveAt()	Xóa một thành phần ở vị trí xác định.
RemoveRange()	Xóa một dãy các thành phần.
Reverse()	Đảo thứ tự các thành phần trong mảng.
SetRange()	Sao chép những thành phần của tập hợp qua dãy những thành phần trong ArrayList.
Sort()	Sắp xếp ArrayList.
ToArray()	Sao chép những thành phần của ArrayList đến một mảng mới.
TrimToSize()	Thiết lập kích thước thật sự chứa các thành phần trong ArrayList

ArrayList

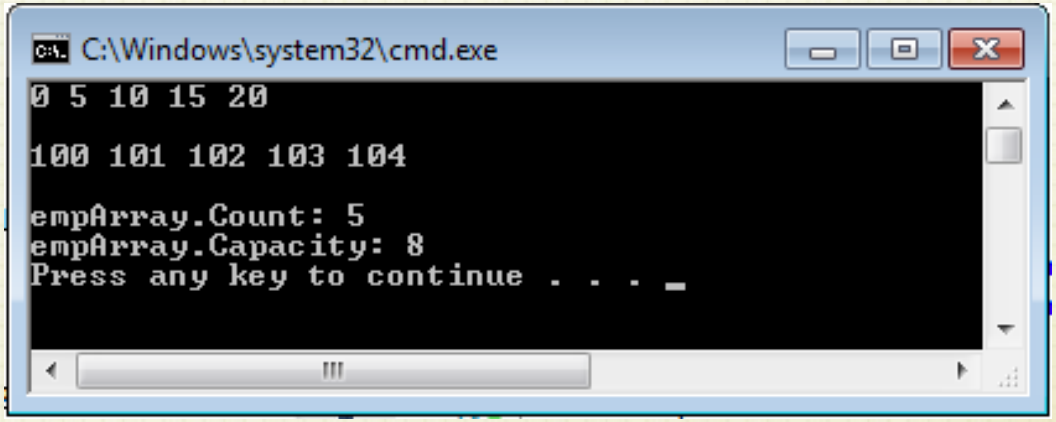
```
01  using System;
02  using System.Collections;
03  public class Employee
04  {
05      private int empID;
06      public Employee(int empID)
07      {
08          this.empID = empID;
09      }
10      public override string ToString()
11      {
12          return empID.ToString();
13      }
14      public int EmpID
15      {
16          get
17          {
18              return empID;
19          }
20          set
21          {
22              empID = value;
23          }
24      }
25  }
```

ArrayList

```
26  class Program
27  {
28      static void Main(string[] args)
29      {
30          ArrayList empArray = new ArrayList();
31          ArrayList intArray = new ArrayList();
32          // đưa vào mảng
33          for (int i = 0; i < 5; i++)
34          {
35              empArray.Add(new Employee(i + 100));
36              intArray.Add(i * 5);
37          }
38          // in tất cả nội dung
39          for (int i = 0; i < intArray.Count; i++)
40          {
41              Console.Write("{0} ", intArray[i].ToString());
42          }
43          Console.WriteLine("\n");
```

ArrayList

```
44          // in tất cả nội dung của mảng
45          for (int i = 0; i < empArray.Count; i++)
46          {
47              Console.Write("{0} ", empArray[i].ToString());
48          }
49          Console.WriteLine("\n");
50          Console.WriteLine("empArray.Count: {0}", empArray.Count);
51          Console.WriteLine("empArray.Capacity: {0}",
    empArray.Capacity);
52      }
53  }
```



Queue (Hàng đợi)

- Hàng đợi là một tập hợp trong đó có thứ tự vào trước và ra trước (FIFO).

Thuộc tính	Mô tả
Count	Thuộc tính trả về số thành phần trong hàng đợi
IsReadOnly	Thuộc tính xác định hàng đợi là chỉ đọc
IsSynchronized	Thuộc tính xác định hàng đợi được đồng bộ
SyncRoot	Thuộc tính trả về đối tượng có thể được sử dụng để đồng bộ truy cập Queue.

Queue (Hàng đợi)

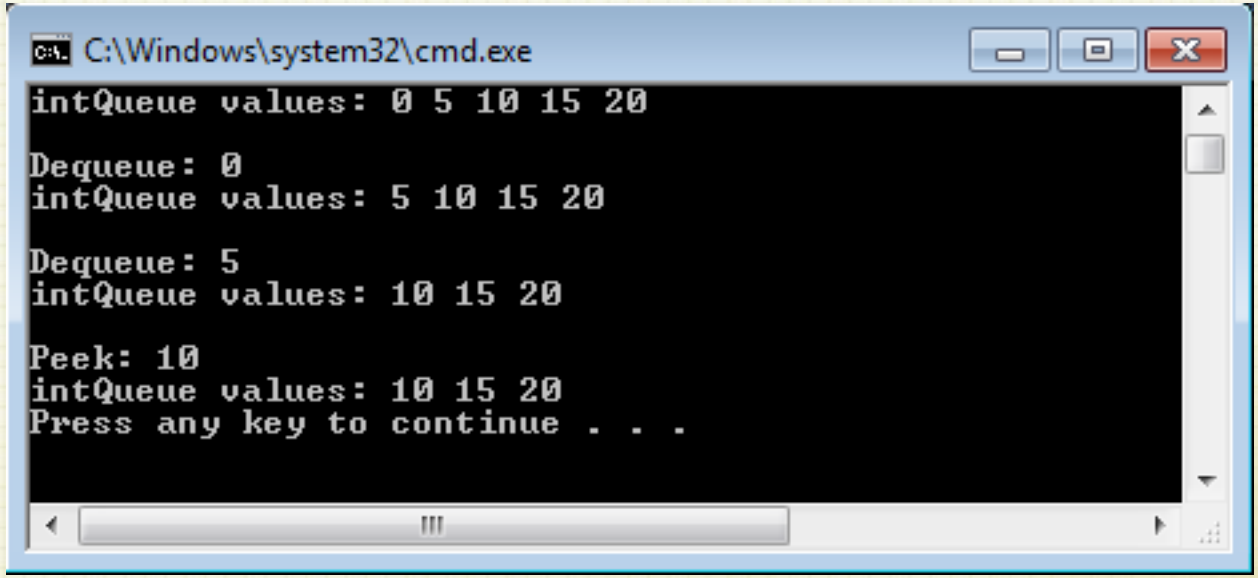
Phương thức	Mô tả
Clear()	Xóa tất cả các thành phần trong hàng đợi
Clone()	Tạo ra một bản sao
Contains()	Xác định xem một thành phần có trong mảng.
CopyTo()	Sao chép những thành phần của hàng đợi đến mảng một chiều đã tồn tại
Dequeue()	Xóa và trả về thành phần bắt đầu của hàng đợi.
Enqueue()	Thêm một thành phần vào hàng đợi.
GetEnumerator()	Trả về một enumerator cho hàng đợi.
Peek()	Trả về phần tử đầu tiên của hàng đợi và không xóa nó.
ToArray()	Sao chép những thành phần qua một mảng mới

Queue (Hàng đợi)

```
using System;
using System.Collections;
class Program
{
    public static void Main()
    {
        Queue intQueue = new Queue();
        // đưa vào trong mảng
        for(int i=0; i <5; i++)
        {
            intQueue.Enqueue(i * 5);
        }
        // hiển thị hàng đợi
        Console.Write("intQueue values: "); PrintValues(intQueue);
        // xóa thành phần ra khỏi hàng đợi
        Console.WriteLine("\nDequeue: {0}", intQueue.Dequeue());
        // hiển thị hàng đợi
        Console.Write("intQueue values: "); PrintValues(intQueue);
        // xóa thành phần khỏi hàng đợi
        Console.WriteLine("\nDequeue: {0}", intQueue.Dequeue());
        // hiển thị hàng đợi
        Console.Write("intQueue values: "); PrintValues(intQueue);
        // Xem thành phần đầu tiên trong hàng đợi.
        Console.WriteLine("\nPeek: {0}", intQueue.Peek());
```

Queue (Hàng đợi)

```
25          // hiển thị hàng đợi
26          Console.Write("intQueue values: "); PrintValues(intQueue);
27      }
28      public static void PrintValues(IEnumerable myCollection)
29      {
30          foreach (Object obj in myCollection)
31              Console.Write("{0} ", obj);
32          Console.WriteLine();
33      }
34  }
```



Stack (Ngăn xếp)

- Ngăn xếp là một tập hợp mà thứ tự là vào trước ra sau hay vào sao ra trước (LIFO)
- Hai phương thức chính cho việc thêm và xóa từ stack là Push và Pop, ngoài ra ngăn xếp cũng đưa ra phương thức Peek tương tự như Peek trong hàng đợi.

Thuộc tính	Mô tả
Count	Thuộc tính trả về số thành phần trong ngăn xếp
IsReadOnly	Thuộc tính xác định ngăn xếp là chỉ đọc
IsSynchronized	Thuộc tính xác định ngăn xếp được đồng bộ
SyncRoot	Thuộc tính trả về đối tượng có thể được sử dụng để đồng bộ truy cập Stack.

Stack (Ngăn xếp)

Phương thức	Mô tả
Clear()	Xóa tất cả các thành phần trong ngăn xếp
Clone()	Tạo ra một bản sao
Contains()	Xác định xem một thành phần có trong mảng.
CopyTo()	Sao chép những thành phần của ngăn xếp đến mảng một chiều đã tồn tại
Pop()	Xóa và trả về phần tử đầu Stack
Push()	Đưa một đối tượng vào đầu ngăn xếp
Peek()	Trả về phần tử đầu tiên của ngăn xếp và không xóa nó.
ToArray()	Sao chép những thành phần qua một mảng mới

Stack (Ngăn xếp)

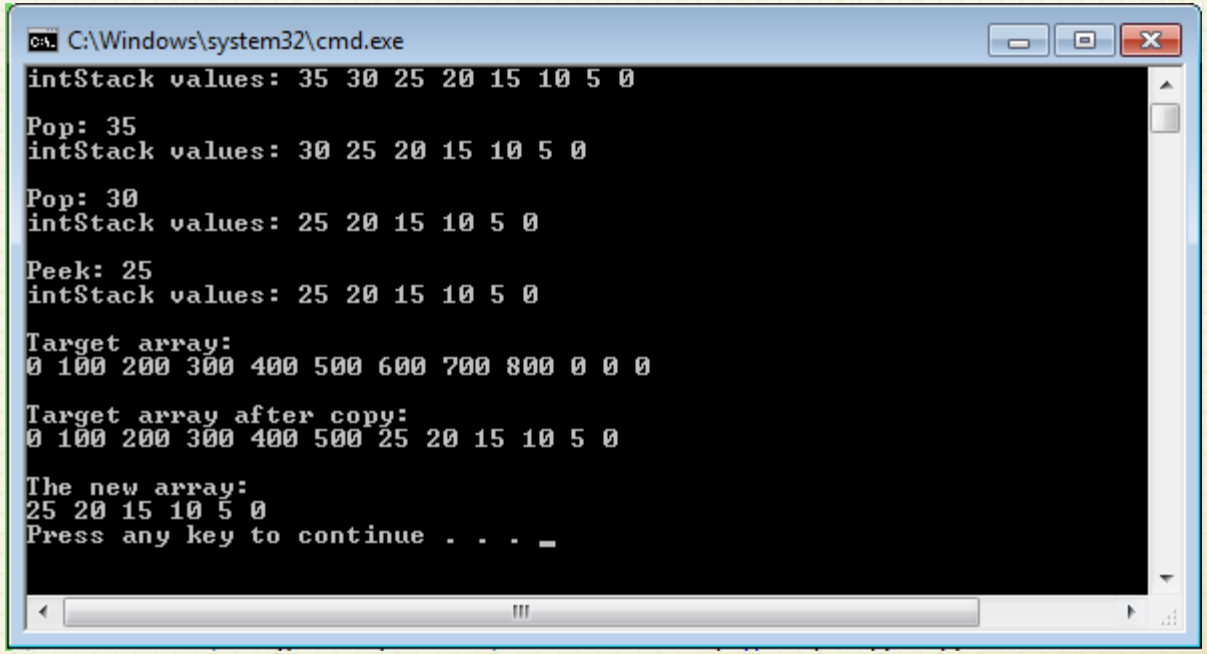
```
01  using System;
02  using System.Collections;
03  public class Program
04  {
05      static void Main()
06      {
07          Stack intStack = new Stack();
08          // đưa vào ngăn xếp
09          for (int i = 0; i < 8; i++)
10          {
11              intStack.Push(i * 5);
12          }
13          // hiển thị stack
14          Console.Write("intStack values: "); PrintValues( intStack );
15          // xóa phần tử đầu tiên
16          Console.WriteLine("\nPop: {0}", intStack.Pop());
17          // hiển thị stack
18          Console.Write("intStack values: "); PrintValues( intStack);
19          // xóa tiếp phần tử khác
20          Console.WriteLine("\nPop: {0}", intStack.Pop());
21          // hiển thị stack
22          Console.Write("intStack values: "); PrintValues( intStack );
```

Stack (Ngăn xếp)

```
23      // xem thành phần đầu tiên stack
24      Console.WriteLine("\nPeek: {0}", intStack.Peek());
25      // hiển thị stack
26      Console.Write("intStack values: "); PrintValues( intStack);
27      // khai báo mảng với 12 phần tử
28      Array targetArray = Array.CreateInstance(typeof(int), 12);
29      for (int i = 0; i <= 8; i++)
30      {
31                  targetArray.SetValue(100 * i, i);
32      }
33      // hiển thị giá trị của mảng
34      Console.WriteLine("\nTarget array: ");
35      PrintValues( targetArray );
35      // chép toàn bộ stack vào mảng tại vị trí 6
36      intStack.CopyTo( targetArray, 6);
37      // hiển thị giá trị của mảng sau copy
38      Console.WriteLine("\nTarget array after copy: ");
39      PrintValues( targetArray );
39      // chép toàn bộ stack vào mảng mới
40      Object[] myArray = intStack.ToArray();
41      // hiển thị giá trị của mảng mới
42      Console.WriteLine("\nThe new array: ");
43      PrintValues(myArray);
43  }
```

Stack (Ngăn xếp)

```
44        public static void PrintValues(IEnumerable myCollection)
45        {
46            foreach (Object obj in myCollection)
47                Console.Write("{0} ", obj);
48            Console.WriteLine();
49        }
50    }
```



Hashtables

- Hashtable là một kiểu từ điển trong đó có hai thành phần chính liên hệ với nhau là key và value
- Hashtable là kiểu từ điển đã được tối ưu cho phép việc truy cập nhanh chóng.

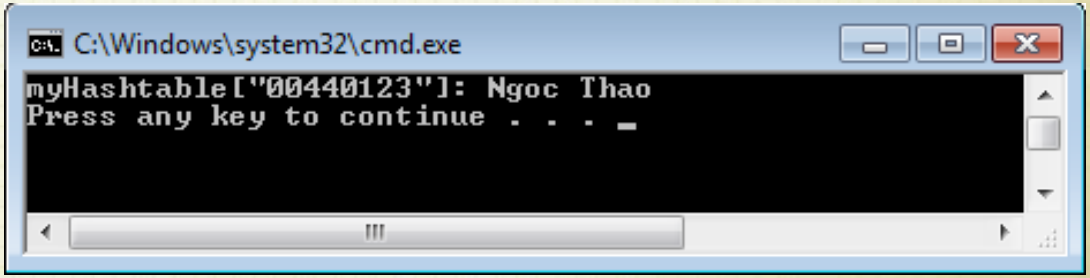
Thuộc tính	Mô tả
Count	Thuộc tính trả về số thành phần trong hashtable
IsReadOnly	Thuộc tính xác định hashtable là chỉ đọc
Keys	Thuộc tính trả về một ICollection chứa những khóa trong hashtable.
Values	Thuộc tính trả về một ICollection chứa những giá trị trong hashtable.

Hashtables

Phương thức	Mô tả
Add()	Thêm một thành phần mới với khóa và giá trị xác định.
Clear()	Xóa tất cả đối tượng trong hashtable.
Item()	Chỉ mục cho hastable
Clone()	Tạo ra một bản sao
Contains()	Xác định xem một thành phần có trong hashtable.
ContainsKey()	Xác định xem hashtable có chứa một khóa xác định
CopyTo()	Sao chép những thành phần của hashtable đến mảng một chiều đã tồn tại
GetEnumerator()	Trả về một enumerator cho hashtable.
Remove()	Xóa một thành phần với khóa xác định.

Hashtables

```
01  using System;
02  using System.Collections;
03  public class Tester
04  {
05      static void Main()
06      {
07          // tạo và khởi tạo hashtable
08          Hashtable hashTable = new Hashtable();
09          hashTable.Add("00440123","Ngoc Thao");
10          hashTable.Add("00123001","My Tien");
11          hashTable.Add("00330124","Thanh Tung");
12          // truy cập qua thuộc tính Item
13          Console.WriteLine("myHashtable[\"00440123\"]:
{0}",  hashTable["00440123"]);
14      }
15  }
16
```



System.IO

- Thư viện System.IO cung cấp nhiều lớp dùng cho việc đọc, ghi file cũng như việc thao tác với file và thư mục.
- Một số lớp chính của System.IO

DriveInfo

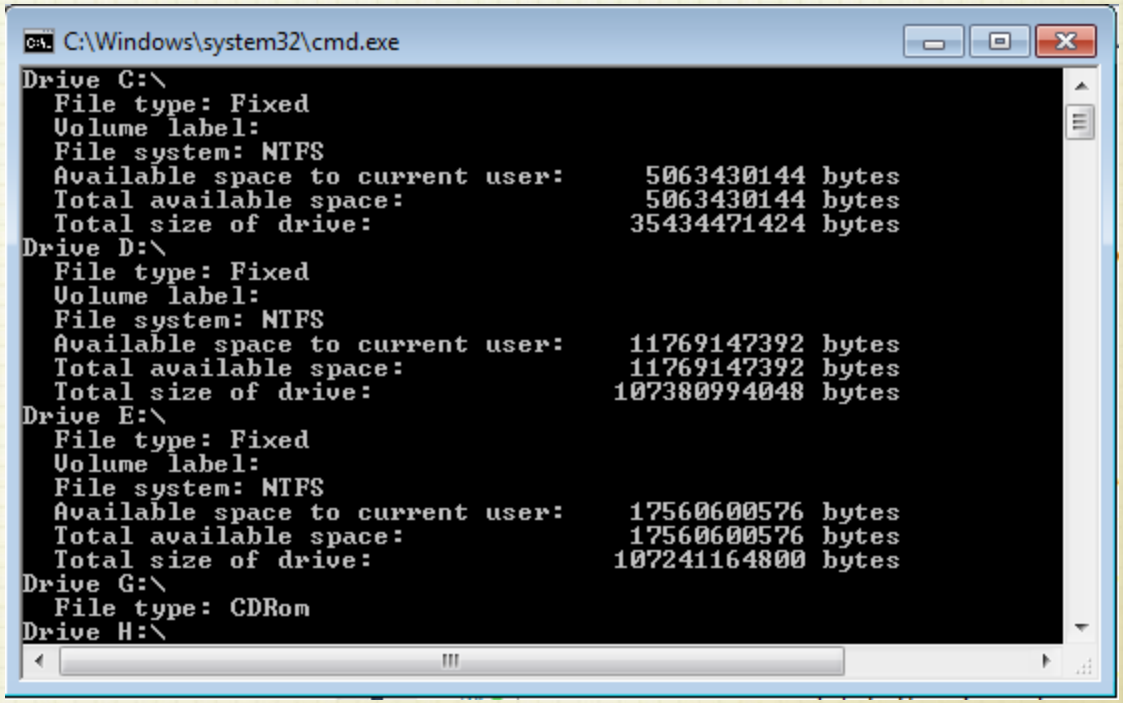
Thuộc tính/ Phương thức cơ bản	Mô tả
DriveFormat	Tên của file system, ví dụ: NTFS/ FAT32
DriveType	Cho biết lọai ổ đĩa. Kiểu dữ liệu trả về là DriveType: CDRom, Fixed, Network, Ram, Removable, Unknown
IsReady	Cho biết ổ đĩa đã sẵn sàng Read/Write.
Name	Tên ổ đĩa
TotalFreeSpace	Xem dung lượng đĩa trống
TotalSize	Xem tổng dung lượng đĩa
GetDrives()	Lấy danh sách ổ đĩa hiện có

DriveInfo

```
using System;
using System.IO;
class Test
{
    public static void Main()
    {
        DriveInfo[] allDrives = DriveInfo.GetDrives();
        foreach (DriveInfo d in allDrives)
        {
            Console.WriteLine("Drive {0}", d.Name);
            Console.WriteLine("  File type: {0}", d.DriveType);
            if (d.IsReady == true)
            {
                Console.WriteLine("  Volume label: {0}", d.VolumeLabel);
                Console.WriteLine("  File system: {0}", d.DriveFormat);
                Console.WriteLine(
                    "  Available space to current user:{0, 15} bytes",
                    d.AvailableFreeSpace);
```

DriveInfo

```
19                  Console.WriteLine(
20                      "  Total available space: {0, 15} bytes",
21                      d.TotalFreeSpace);
22                  Console.WriteLine(
23                      "  Total size of drive: {0, 15} bytes",
24                      d.TotalSize);
25              }
26          }
27      }
28  }
```



DirectoryInfo

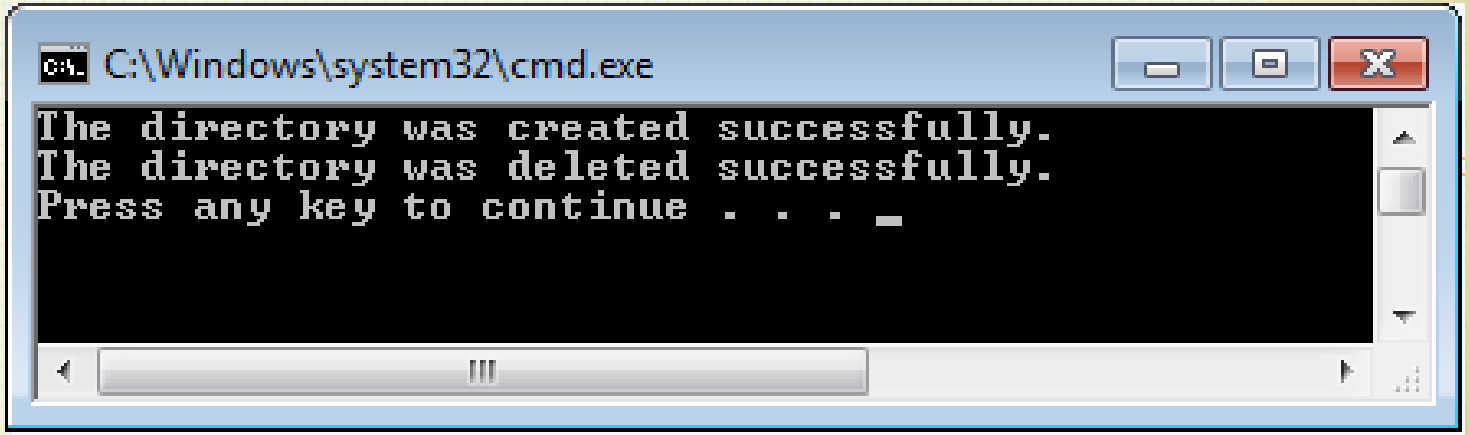
Thuộc tính/ phương thức cơ bản	Mô tả
CreationTime	Xem hoặc thiết lập thời gian tạo thư mục
Exists	Kiểm tra thư mục tồn tại trên ổ đĩa
FullName	Lấy đường dẫn của tới thư mục
LastAccessTime	Cho biết thời gian cuối cùng thư mục (file) được truy cập
Name	Cho biết tên của thư mục
Parent	Trả về thư mục cha.
FileAttributes Attributes	Cho biết thuộc tính của thự mục (file) FileAttributes là 1 enum gồm các giá trị như: Directory, Readonly, Hidden,…
Create()	Tạo thư mục
Delete()	Xóa thư mục
MoveTo()	Di chuyển thư mục
GetDirectories()	Lấy các thư mục con trong thư mục
GetFiles	Lấy tất cả các tập tin trong thư mục

DirectoryInfo

```
01  using System;
02  using System.IO;
03  class Test
04  {
05      public static void Main()
06      {
07          // Specify the directories you want to manipulate.
08          DirectoryInfo di = new DirectoryInfo(@"c:\MyDir");
09          try
10          {
11              // Determine whether the directory exists.
12              if (di.Exists)
13              {
14                  // Indicate that the directory already exists.
15                  Console.WriteLine("That path exists already.");
16                  return;
17              }
18              // Try to create the directory.
19              di.Create();
20              Console.WriteLine("The directory was created
    successfully.");
```

DirectoryInfo

```
21            // Delete the directory.
22            di.Delete();
23            Console.WriteLine("The directory was deleted successfully.");
24        }
25        catch (Exception e)
26        {
27            Console.WriteLine("The process failed: {0}",e.ToString());
28        }
29        finally
30        {
31        }
30    }
31 }
```



FileInfo

Thuộc tính/Phương thức cơ bản	Mô tả
CreationTime	Xem hoặc thiết lập thời gian tạo thư mục
Exists	Kiểm tra thư mục tồn tại trên ổ đĩa
Directory	Trả về đối tượng thư mục cha
DirectoryName	Trả về chuỗi đường dẫn (full path) của thư mục cha
Extension	Trả về tên đuôi file (txt,bat,exe,…)
Name	Cho biết tên của file
Attributes	Cho biết thuộc tính của file
CopyTo()	Copy file đến 1 nơi khác
Create()	Tạo file
Delete()	Xóa file
MoveTo()	Di chuyển file hoặc đổi tên file
CreateText	Tạo StreamWriter để ghi file
OpenText	Tạo StreamReader để đọc file
ReplaceFile	Thay đổi nội dung file

FileInfo

```
01  using System;
02  using System.IO;
03  class Test
04  {
05      public static void Main()
06      {
07          string path = Path.GetTempFileName();
08          FileInfo fi1 = new FileInfo(path);
09          //Create a file to write to.
10          using (StreamWriter sw = fi1.CreateText())
11          {
12              sw.WriteLine("Hello");
13              sw.WriteLine("And");
14              sw.WriteLine("Welcome");
15          }
16          //Open the file to read from.
17          using (StreamReader sr = fi1.OpenText())
18          {
19              string s = "";
20              while ((s = sr.ReadLine()) != null)
21              {
22                  Console.WriteLine(s);
23              }
24          }
```

FileInfo

```
25          try
26          {
27              string path2 = Path.GetTempFileName();
28              FileInfo fi2 = new FileInfo(path2);
29              //Ensure that the target does not exist.
30              fi2.Delete();
31              //Copy the file.
32              fi1.CopyTo(path2);
33              Console.WriteLine("{0} was copied to {1}.", path, path2);
34              //Delete the newly created file.
35              fi2.Delete();
36              Console.WriteLine("{0} was successfully deleted.", path2);
37          }
38          catch (Exception e)
39          {
40              Console.WriteLine("The process failed: {0}",e.ToString());
41          }
42      }
43  }
```

Xử lí đọc/ghi file

- Đọc và ghi dữ liệu sẽ được thực hiện thông qua lớp Stream.
- Stream là 1 luồng dữ liệu, nó đưa dữ liệu từ điểm bắt đầu đến điểm cuối.
- System.IO.Stream là một lớp abstract định nghĩa một số thành viên có khả năng hỗ trợ việc đọc/ghi đồng bộ (synchronus) hoặc không đồng bộ (asynchronous) đối với khối trữ tin.

Stream Class

Thuộc tính/Phương thức cơ bản	Mô tả
CanRead	Luồng có hỗ trợ đọc
CanSeek	Luồn có hỗ trợ di chuyển con trỏ
CanTimeOut	Xác định xem luồng có timeout hay không
CanWrite	Luồng có hỗ trợ ghi
Length	Chiều dài (theo bytes) của luồng
ReadTimeout	Thiết lập timeout cho phương thức Read
WriteTimeout	Thiết lập timeout cho phương thức Write
Position	Lấy hoặc xác lập vị trí con trỏ trong luồng
Close()	Đóng luồng và giải phóng tài nguôn
Flush()	Đẩy toàn bộ dữ liệu buffer trong luồng lên trên thiết bị
Read()	Thực thi phương thức đọc mảng byte trên luồng.
Seek()	Di chuyển vị trí con trỏ đọc
Write()	Ghi mảng byte lên trên luồng

FileStream Class

- Lớp FileStream là lớp dẫn xuất từ lớp Stream. FileStream có một số phương thức và thuộc tính riêng.

Thuộc tính/Phương thức cơ bản	Mô tả
Name	Lấy tên của file
Lock()	Khóa file, tránh truy xuất đồng thời lên File
Unlock	Mở khóa file, có thể truy xuất đồng thời lên file

StreamReader

- StreamReader có thể dùng để đọc văn bản

Thuộc Tính	Mô tả
BaseStream	Trả về luồng đọc
CurrentEncoding	Lấy thông tin định dạng của luồng đang sử dụng
EndOfStream	Xác định con trỏ đọc đến cuối luồng chưa

Phương thức	Mô tả
Close	Đóng luồng và giải phóng tài nguyên
Peek	Trả về giá trị kí tự tiếp theo trong luồng, không di chuyển con trỏ đọc
Read	Thực thi phương thức đọc mảng các kí tự trên luồng.
ReadBlock	Đọc khối kí tự tiếp theo trên luồng.
ReadLine	Đọc nguyên dòng trên luồng
ReadToEnd	Đọc tất cả các kí tự tới cuối luồng

Ví dụ

```
01  using System;
02  using System.IO;
03  class Test
04  {
05      public static void Main()
06      {
07          try
08          {
09              // Tạo một StreamReader để đọc file
10              using (StreamReader sr = new StreamReader("TestFile.txt"))
11              {
12                  string line;
13                  // Đọc từng dòng của File
14                  while ((line = sr.ReadLine()) != null)
15                  {
16                      Console.WriteLine(line);
17                  }
18              }
19          }
20          catch (Exception e){
21              // Hiển thị thông điệp lỗi
22              Console.WriteLine("The file could not be read:");
23              Console.WriteLine(e.Message);
24          }
25      }
26  }
```

StreamWriter

- StreamWriter có thể dùng để ghi văn bản

Thuộc tính	Mô tả
AutoFlush	Thiết lập cơ chế tự động Flush, sau mỗi lệnh Write
BaseStream	Trả về luồng bên dưới
Encoding	Lấy chế độ mã hóa hiện hành của luồng

Phương thức	Mô tả
Close	Đóng luồng và giải phóng tài nguyên
Write	Ghi vào luồng
WriteLine	Ghi một chuỗi kí tự vào luồng và xuống hàng

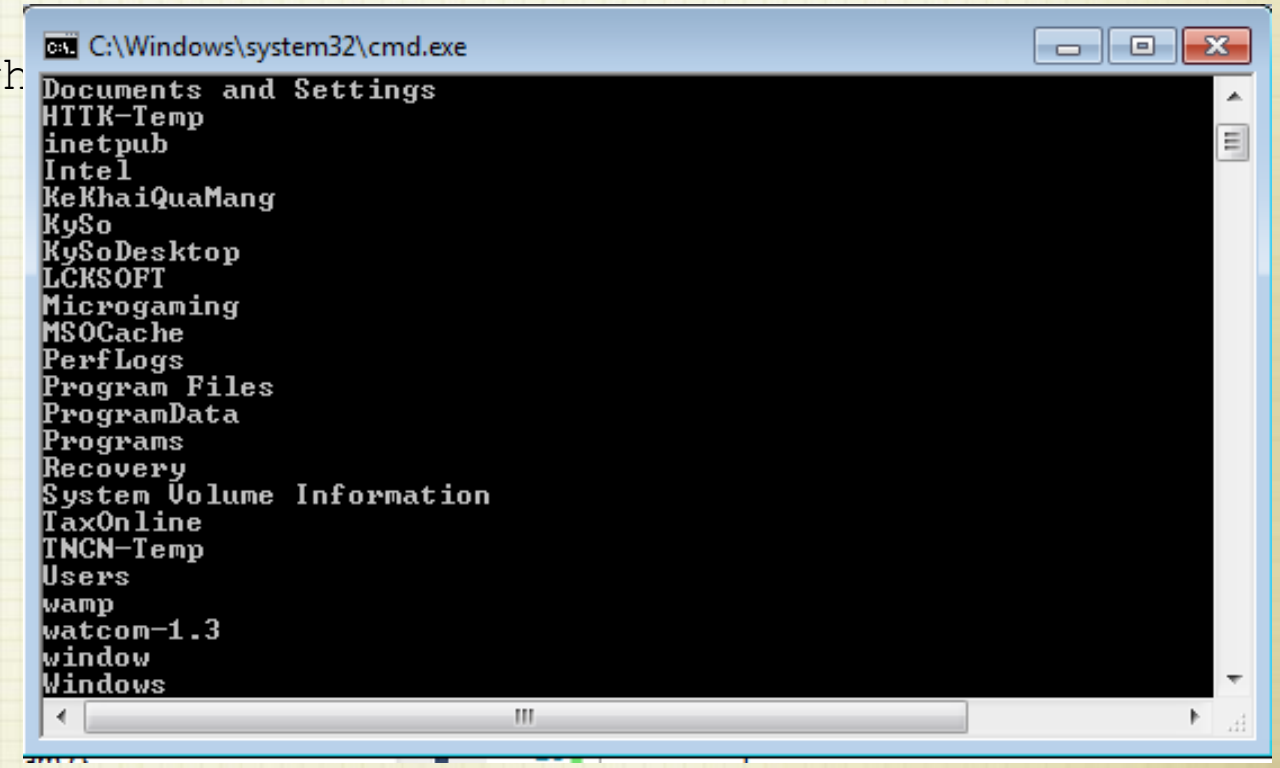
Ví dụ

```
using System;
using System.IO;
class Program
{
    static void Main(string[] args)
    {
        // Lấy các thư mục hiện hành trên ổ đĩa
        DirectoryInfo[] cDirs = new
DirectoryInfo(@"c:\").GetDirectories();
        // Viết tên các thư mục vào file
        using (StreamWriter sw = new
                        StreamWriter("CDriveDirs.txt"))
        {
            foreach (DirectoryInfo dir in cDirs)
            {
                sw.WriteLine(dir.Name);

            }
        }
```

Ví dụ

```
18          // Đọc và hiển thi tên thư mục từ file
19          string line = "";
20          using (StreamReader sr = new
    StreamReader("CDriveDirs.txt"))
21          {
22              wh
23              {
24
25              }
26          }
27      }
28  }
29
```



BinaryReader và BinaryWriter

- Tương tự như StreamReader và StreamWriter, BinaryReader và BinaryWriter có thể dùng để đọc file nhị phân.

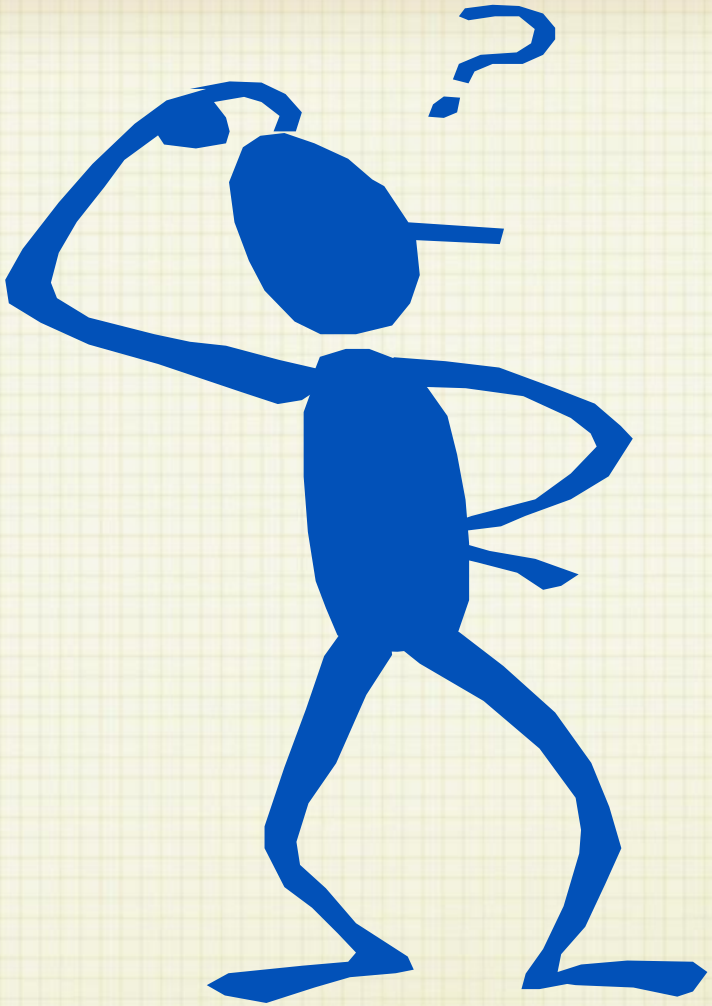
```
FileStream theFile =
File.Open(@"c:\somefile.bin", FileMode.Open);
BinaryReader reader = new BinaryReader(theFile);
long number = reader.ReadInt64(); byte[] bytes =
reader.ReadBytes(4); string s = reader.ReadString();
reader.Close();

FileStream theFile = File.Open(@"c:\somefile.bin",
       FileMode.OpenOrCreate, FileAccess.Write);
BinaryWriter writer = new BinaryWriter(theFile);
long number = 100;

byte[] bytes = new byte[] { 10, 20, 50, 100 };
string s = "Toi di hoc";
writer.Write(number); writer.Write(bytes);
writer.Write(s);
```

BufferedStream

- BufferedStream thường được sử dụng để tăng hiệu quả đọc ghi dữ liệu.

```
FileStream newFile = File.Create(@"c:\test.txt");
BufferedStream buffered = new BufferedStream(newFile);
StreamWriter writer = new StreamWriter(buffered);
writer.WriteLine("Some data");
streamWriter.Close();
bufferedStream.Close();
fileStream.Close();
```

Q & A